

Ingegneria del Software

Marco Tartaglione

7 ottobre 2026

0 Indice

I	Primo modulo	4
1	Introduzione	4
1.1	Fattori di qualità di un buon software	4
2	Principi e Concetti Object-Oriented	5
2.1	Come viene implementata l'ereditarietà multipla	5
2.2	Si esegua una classificazione del polimorfismo secondo Cardelli-Wegner e si mostri l'implementazione del polimorfismo per inclusione.	5
2.3	ADT ed incapsulamento	6
3	Net	7
3.1	Procedimento di compilazione ed esecuzione del codice all'interno del framework .NET tramite il CLR	7
3.2	Framework .NET	8
3.3	Tecnologia COM	8
3.4	Tipi di dati in .NET	9
3.5	Differenza tra tipi valore e tipi riferimento in .NET	10
3.6	Garbage Collector in C#	11
3.7	Ciclo di vita di un oggetto esemplificandone i passi	12
3.8	Passaggio dei parametri in C#	13
3.9	Differenze tra interfaccia e classe astratta	14
3.10	Concetto di delegato in C#	15
3.11	Concetto di evento in C#	16
3.12	Metaprogrammazione e riflessione in C#	16
4	Design Principle	17
4.1	Spiegare i quattro bad design (fragilità, immobilità, rigidità, viscosità)	17
4.2	Principio di singola responsabilità con almeno un esempio	18
4.3	Principio di inversione delle dipendenze con almeno un esempio	19
4.4	Principio di segregazione delle interfacce con almeno un esempio	20
4.5	Principio aperto/chiuso con almeno un esempio	20
4.6	Principio di sostituibilità di Liskov con almeno un esempio	21
4.7	Principi per l'architettura dei package	22
5	Design patterns	22
5.1	Pattern Singleton con esempi	23
5.2	Pattern Observer con esempi	23
5.3	Pattern Flyweight con esempi	24
5.4	Pattern Strategy con esempi	24
5.5	Pattern Adapter con esempi	25
5.6	Pattern Decorator con esempi	25
5.7	Pattern State con esempi	26
5.8	Pattern Composite con esempi	26
5.9	Pattern Visitor con esempi	27
5.10	Pattern Abstract Factory con esempi	28

5.11	Pattern MVP ed MVC	29
6	Version Control System	30
6.1	Modello LMU nei VCS con vantaggi e svantaggi	30
6.2	Modello CMM nei VCS con vantaggi e svantaggi	30
II	Secondo modulo	32
7	Modelli, Linguaggi e Modelli di Processo di sviluppo	32
7.1	Il modello a cascata e le sue criticità	32
7.2	Il modello a cascata e il modello iterativo	33
7.3	Il modello di processo RUP	33
8	Analisi dei requisiti	35
8.1	Tipologie di requisiti	35
9	Analisi dei requisiti di sicurezza	36
9.1	Principali categorie di requisiti per la sicurezza	36
9.2	Il ciclo di vita della valutazione del rischio	36
10	Progettazione per la sicurezza	37
10.1	Linee guida di progettazione nella sicurezza	37
10.2	Test di sicurezza: Black-box e White-box	39
10.3	Capacità di sopravvivenza del sistema	39

Primo modulo

1 Introduzione

1.1 Fattori di qualità di un buon software

La qualità di un software non si limita alla sua mera esecuzione, ma si valuta attraverso una serie di attributi sistematici. Questi fattori vengono tradizionalmente suddivisi in **qualità esterne** (percepibili dall'utente finale) e **qualità interne** (percepibili dagli sviluppatori e cruciali per la manutenzione).

- **Correttezza (External):** Capacità del sistema software di svolgere esattamente i compiti definiti dalle specifiche dei requisiti.
- **Robustezza (External):** Capacità del sistema di gestire correttamente situazioni anomale o input non specificati nei requisiti (es. malfunzionamenti hardware, input errati dell'utente), evitando fallimenti catastrofici o perdite di dati.
- **Affidabilità (Reliability - External):** Probabilità che il software esegua le sue funzioni senza guasti in un determinato intervallo di tempo e sotto specifiche condizioni. Dipende strettamente da correttezza e robustezza.
- **Usabilità (Facilità d'uso - External):** Facilità con cui gli utenti possono apprendere l'utilizzo del software, inserire l'input e interpretare l'output del sistema.
- **Efficienza (External/Internal):** Capacità del software di ottimizzare l'uso delle risorse hardware (tempo di CPU, memoria, banda di rete) durante la sua esecuzione.
- **Estendibilità (Manutenibilità - Internal):** Facilità con cui il software può essere modificato per adattarsi a nuovi requisiti o evoluzioni del contesto. Si basa su due principi cardine:
 - *Semplicità architetturale:* design lineare e privo di complessità superflue.
 - *Modularità e decentralizzazione:* divisione del sistema in componenti autonome, coese e debolmente accoppiate.
- **Riusabilità (Internal):** Attitudine di moduli o architetture software a essere impiegati per la costruzione di applicazioni differenti da quella per cui sono stati originariamente progettati.
- **Verificabilità (Testabilità - Internal):** Facilità con cui è possibile stabilire la correttezza del software attraverso l'esecuzione di test (es. unit test, test d'integrazione).

- **Portabilità (Internal/External):** Facilità con cui il software può essere trasferito da un ambiente hardware/software (sistema operativo, DBMS) a un altro.

2 Principi e Concetti Object-Oriented

2.1 Come viene implementata l'ereditarietà multipla

A seconda della struttura della gerarchia, l'ereditarietà si classifica in due tipologie:

- **Ereditarietà semplice:** Ogni classe della gerarchia deriva da al più una superclasse (vincolo strutturale rigido in linguaggi come Java e .NET a meno delle interfacce). La struttura risultante è sempre rappresentabile mediante un **albero**.
- **Ereditarietà multipla:** Si verifica quando almeno una classe della gerarchia deriva contemporaneamente da due o più superclassi (meccanismo supportato, ad esempio, in C++).

L'introduzione dell'ereditarietà multipla complica notevolmente la gestione del codice. Se all'interno della gerarchia esistono degli antenati comuni, la struttura logica non è più lineare ma si trasforma in un **reticolo**. Questa conformazione introduce due problematiche critiche:

1. **Conflitti di nome:** Ambiguità generate nel momento in cui due o più superclassi espongono attributi o metodi con la medesima firma, rendendo equivoca l'invocazione da parte della sottoclasse.
2. **Complessità di gestione del layout:** Difficoltà del compilatore nel mappare la struttura fisica dei dati in memoria e nel risolvere l'ordine di accesso alle tabelle dei metodi virtuali.

A causa di tali complessità operative, l'ingegneria del software suggerisce spesso di effettuare una trasformazione architetturale in fase di progettazione, convertendo i modelli a ereditarietà multipla in modelli a **ereditarietà semplice**. Le tecniche principali fornite dai moderni ambienti di sviluppo includono:

- **Interfacce:** Scomposizione delle superclassi in tipi puri di specifica (che non ereditano implementazione o stato), eliminando i conflitti di layout in memoria.
- **Composizione e delega:** Una tecnica alternativa in cui la classe instaura una relazione di tipo "contenitore/contenuto" (Has-A) anziché "Is-A", includendo le istanze delle vecchie superclassi nella propria struttura dati e delegando loro lo svolgimento delle funzionalità richieste.

2.2 Si esegua una classificazione del polimorfismo secondo Cardelli-Wegner e si mostri l'implementazione del polimorfismo per inclusione.

Secondo la tassonomia di Cardelli-Wegner, il polimorfismo esprime la capacità di un'entità software di assumere più forme ed è classificabile secondo una struttura gerarchica ad albero:

- **Polimorfismo Universale:** Esegue le medesime operazioni su tipi diversi, con una struttura potenzialmente infinita di tipi comuni.

- **Parametrico:** Il codice viene scritto prescindendo da un tipo specifico, che viene passato come parametro esplicito (es. i *Generics* in C o i *Template* in C++).
- **Per inclusione (o sottotipo):** Un oggetto può appartenere a più tipi contemporaneamente, legati da una relazione di sottotipazione (ereditarietà). Il codice opera sul tipo base ma manipola istanze delle sottoclassi.
- **Polimorfismo Ad-Hoc:** Agisce su un insieme finito di tipi, spesso disgiunti, e il comportamento dell'operazione può cambiare radicalmente a seconda del tipo.
 - **Overloading (Sovraccaricamento):** Lo stesso identificatore denota funzioni diverse. La risoluzione avviene a tempo di compilazione in base alla firma (argomenti) del metodo.
 - **Coercion (Coercizione):** Un'operazione semantica che converte automaticamente un argomento nel tipo atteso dall'operatore per evitare un errore (es. la conversione implicita da `int` a `float`).

Il polimorfismo per inclusione si basa sulla gerarchia delle classi (relazione *Is-A*) e sul meccanismo di **late-binding**:

```

1 public class A {
2     public virtual void Func1(){...}
3 }
4
5 public class B : A {
6     public override void Func1(){...}
7     public virtual void Func2(){...}
8 }

```

Il funzionamento del polimorfismo per inclusione si articola attraverso passaggi ben definiti a runtime:

1. **Assegnamento compatibile (Sottotipazione):** Grazie alla relazione di ereditarietà, un riferimento statico del tipo della superclasse (A) può memorizzare l'indirizzo di un oggetto istanziato dinamicamente dalla sottoclasse (B), ad esempio tramite un'istruzione del tipo `A x = new B();`.
2. **Risoluzione tramite Late-Binding:** Quando sul riferimento viene invocato il metodo (`x.Func1()`), il compilatore non associa staticamente la chiamata al codice della classe A. Al contrario, la risoluzione viene rimandata a runtime.
3. **Interrogazione della Virtual Method Table (VMT):** Il sistema runtime segue il puntatore dell'istanza reale allocata nello Heap per accedere alla tabella dei metodi virtuali specifica della classe concreta (B). Poiché la classe B ha effettuato l'override di `Func1()`, la tabella conterrà il puntatore all'implementazione ridefinita.

Di conseguenza, viene eseguito il comportamento specifico della sottoclasse, garantendo che il software risponda in modo polimorfo in base alla natura dell'oggetto reale e non al tipo della variabile che lo contiene.

2.3 ADT ed incapsulamento

Un **Abstract Data Type (ADT)** è un modello definito da una collezione di valori e da un insieme di operazioni ammissibili su tali valori, prescindendo totalmente dalla loro rappresentazione in memoria.

La specifica di un ADT si divide in due componenti:

- **Interfaccia:** Rappresenta la vista pubblica dell'ADT. Definisce la sintassi e la semantica delle operazioni applicabili (i metodi pubblici).

- **Implementazione:** Rappresenta la vista privata dell'ADT, ovvero come i dati vengono effettivamente memorizzati e come le operazioni vengono codificate (strutture dati interne, attributi e metodi privati).

L'ADT sfrutta due principi cardine dell'ingegneria del software:

1. **Incapsulamento:** Il meccanismo che permette di raggruppare in un'unica entità logica i dati e le funzioni che operano su di essi.
2. **Information Hiding (Principio di Parnas):** La politica di design che consiste nel nascondere i dettagli implementativi e le scelte progettuali sensibili dietro l'interfaccia pubblica.

Grazie a questa separazione, i moduli clienti dipendono esclusivamente dall'interfaccia stabile dell'ADT. Ciò garantisce l'indipendenza dalla rappresentazione: l'implementazione interna può essere ottimizzata, modificata o sostituita in totale sicurezza, minimizzando l'impatto di regressione e i costi di manutenzione sull'intero sistema software.

3 Net

3.1 Procedimento di compilazione ed esecuzione del codice all'interno del framework .NET tramite il CLR

Il procedimento di compilazione ed esecuzione di codice nel framework .NET si articola in sei fasi:

- **Sorgenti:** il codice scritto dallo sviluppatore in uno dei linguaggi supportati dalla piattaforma .NET (ad es. C#, F#, VB.NET). Ciascun linguaggio dispone di un proprio compilatore, ma tutti condividono lo stesso modello di esecuzione sottostante grazie alla Common Type System (CTS), che definisce le regole comuni sui tipi di dato.
- **Compilatore .NET:** il compilatore specifico del linguaggio traduce il codice sorgente in codice intermedio, effettuando in questa fase i controlli sintattici e semantici tipici della compilazione.
- **Codice IL:** l'output della compilazione è un assembly (eventualmente composto da più file) espresso in *Common Intermediate Language* (CIL o IL), un linguaggio indipendente dalla piattaforma e dal linguaggio di origine. L'assembly contiene, oltre al codice IL, i metadati che descrivono in modo completo tipi, membri, riferimenti e versioning del codice stesso. Gli assembly possono essere privati (utilizzabili da una sola applicazione), condivisi (registrati nella Global Assembly Cache e disponibili a più applicazioni) oppure scaricati da un URL.
- **Compilatore JIT:** il *Just-In-Time compiler* traduce il codice IL in codice macchina specifico per l'architettura del processore su cui il programma viene eseguito, operazione che avviene a runtime e tipicamente metodo per metodo, al momento della prima invocazione (*lazy compilation*).
- **Codice nativo:** il codice macchina prodotto dal JIT, direttamente eseguibile dal processore. A differenza del codice IL, non è portabile tra architetture diverse, ma è specifico per la piattaforma su cui è stato generato.
- **Esecuzione:** il codice nativo viene eseguito sotto il controllo del CLR, che ne supervisiona il comportamento per l'intera durata del processo (gestione della memoria, sicurezza, gestione delle eccezioni, threading).

Oltre a gestire il ciclo di compilazione/esecuzione appena descritto, la Common Language Runtime (CLR) offre una serie di servizi infrastrutturali agli applicativi, tra cui:

- **Garbage Collector:** gestisce automaticamente l'allocazione e la liberazione della memoria sul Managed Heap, deallocando gli oggetti non più referenziati senza intervento esplicito del programmatore.
- **Funzionalità di libreria già pronte all'uso:** un ampio insieme di servizi forniti dalla Base Class Library (BCL), come operazioni di I/O su file, gestione delle eccezioni, accesso alla rete, collezioni e strutture dati, sicurezza e supporto al multithreading.

3.2 Framework .NET

Il Framework .NET è un ambiente di esecuzione gestito, provvisto di una vasta libreria di classi integrata. Nasce con l'obiettivo di semplificare lo sviluppo e il deployment delle applicazioni, aumentare l'affidabilità del codice e unificare il modello di programmazione tra diverse tecnologie. Il framework è fortemente integrato con il modello COM (*Component Object Model*) ed è nativamente orientato agli oggetti (*object-oriented*). Riduce drasticamente i più comuni errori di programmazione grazie a meccanismi automatici quali il *Garbage Collector* per la gestione della memoria, un rigido *Type Checker* e una gestione strutturata delle eccezioni. Le funzionalità core di .NET sono disponibili in modo agnostico rispetto al linguaggio utilizzato; ciò permette la scrittura di parti diverse dello stesso sistema in linguaggi differenti (es. C#, F#, VB.NET), supportando pienamente l'ereditarietà cross-linguaggio.

Dal punto di vista architetturale, .NET rappresenta un'implementazione della specifica CLI (*Common Language Infrastructure*) ed è composto principalmente da:

- **IL (Intermediate Language):** Il codice sorgente di alto livello viene inizialmente compilato in un linguaggio intermedio indipendente dalla CPU (noto anche come MSIL o CIL).
- **CLR (Common Language Runtime):** Il motore di esecuzione del framework che si occupa di gestire l'esecuzione del codice IL, l'allocazione della memoria, la sicurezza e la sincronizzazione dei thread.
- **CTS (Common Type System):** Una specifica che definisce come i tipi di dati sono dichiarati, utilizzati e gestiti nel runtime, garantendo l'interoperabilità tra linguaggi diversi. Ad esempio, impone regole rigide affinché le classi siano definite in PascalCase (es. `VisualizzatoreLog`) e i relativi metodi in camelCase (es. `inviaDati()`).
- **CLS (Common Language Specification):** Un sottoinsieme di regole del CTS a cui i produttori di compilatori devono attenersi per garantire che gli oggetti creati in un linguaggio possano essere usati senza problemi da un altro.

L'ambiente di esecuzione si completa con un compilatore *JIT (Just-In-Time)*. Il JIT riceve in input gli *assembly* contenenti il codice IL e, solo al momento dell'effettiva esecuzione del programma, li traduce dinamicamente in codice macchina nativo ottimizzato per la specifica architettura hardware ospite.

3.3 Tecnologia COM

La tecnologia *Component Object Model* (COM) è un'architettura proprietaria sviluppata da Microsoft per la creazione di componenti software binari riusabili, interoperabili e indipendenti dal linguaggio di programmazione utilizzato. COM non costituisce un linguaggio a sé stante, bensì uno standard binario che specifica un

modello a oggetti rigoroso e un insieme di requisiti di programmazione. Questi permettono a moduli software differenti (come file `.dll` o `.exe`), anche sviluppati da venditori diversi o in epoche diverse, di interagire dinamicamente tra loro all'interno dello stesso spazio di indirizzamento o attraverso la rete (DCOM).

L'architettura si fonda su specifiche regole di gestione della memoria, del ciclo di vita degli oggetti e del riuso del codice:

- **Gestione della memoria tramite Reference Counting:** Il ciclo di vita di un oggetto COM è regolato internamente dall'oggetto stesso attraverso la tecnica del conteggio dei riferimenti (*Reference Counting*), esposta tramite i metodi dell'interfaccia radice `IUnknown` (`AddRef` e `Release`). L'oggetto è il solo responsabile della liberazione della propria memoria non appena il contatore dei riferimenti attivi scende a zero. Questo approccio introduce una severa criticità in presenza di *riferimenti circolari*.
- **Riuso del codice senza ereditarietà classica:** COM non supporta l'ereditarietà dell'implementazione (tipica della programmazione orientata agli oggetti tradizionale). Il riuso del codice e l'estensione delle funzionalità avvengono esclusivamente a livello binario mediante due pattern strutturali:
 - *Contenimento/Composizione*.
 - *Delega (Aggregazione)*.
- **Il Registro di Windows e il deployment:** I componenti COM, identificati univocamente da identificatori globali (GUID), devono essere registrati tassativamente all'interno del Registro di sistema di Windows. Per ciascun componente può esistere una sola versione attiva globalmente associata a quel GUID nel registro. Questa limitazione architetturale ha storicamente reso il deployment e l'aggiornamento delle applicazioni estremamente complessi e fragili, dando origine al fenomeno noto come *Inferno delle DLL (DLL Hell)*, in cui l'installazione di una nuova versione di un componente software sovrascriveva e corrompeva il funzionamento di applicazioni preesistenti e dipendenti da una versione precedente dello stesso file.

3.4 Tipi di dati in .NET

L'infrastruttura del Common Language Runtime (CLR) di .NET organizza il sistema dei tipi attraverso il *Common Type System* (CTS). Ogni entità programmabile appartiene a una categoria ben definita, differenziandosi principalmente per la modalità di allocazione della memoria, per le strategie di passaggio dei parametri (per valore o per riferimento) e per il ciclo di vita all'interno dei segmenti di memoria dello *Stack* e dello *Heap*.

I tipi di dati messi a disposizione si articolano in due macro-categorie fondamentali, all'interno delle quali si posizionano costrutti specifici:

- **Value Type:** Contengono direttamente i propri dati all'interno dello spazio di memoria allocato. Quando un tipo per valore viene copiato o passato a un metodo, viene creata una copia bit a bit indipendente del dato. Non possono essere `null` (a meno che non siano dichiarati come *Nullable*) e non supportano l'ereditarietà classica. Di questa categoria fanno parte:
 - *Tipi Primitivi/Predefiniti:* Tipi numerici integrati (come `int`, `double`, `decimal`), booleani (`bool`) e caratteri (`char`).
 - *Strutture (Struct):* Costrutti personalizzati usati per modellare vettori di dati correlati.
 - *Enumerativi (Enum):* Tipi a valore speciale che racchiudono un insieme di costanti.
- **Reference Type:** Memorizzano nello *Stack* esclusivamente un indirizzo di memoria (puntatore) che fa riferimento all'area in cui risiedono i dati effettivi, allocata dinamicamente all'interno del *Managed Heap*. La copia o il passaggio di un reference type duplica solo il riferimento, facendo sì che più variabili puntino alla stessa identica istanza. Il loro ciclo di vita e la deallocazione sono interamente gestiti dal *Garbage Collector*. Di questa categoria fanno parte:

- *Classi*: I mattoni fondamentali della programmazione a oggetti in .NET. Supportano l’ereditarietà singola, l’incapsulamento e il polimorfismo, permettendo la definizione di metodi, proprietà ed eventi.
- *Interfacce*: Contratti puramente astratti che definiscono firme di metodi, proprietà o eventi senza fornire l’implementazione. Consentono il disaccoppiamento architetturale e l’ereditarietà multipla di comportamento.
- *Delegati (Delegate)*: Tipi di riferimento che incapsulano un puntatore a un metodo con una specifica firma.
- *Array*: Collezioni sequenziali e a dimensione fissa di elementi dello stesso tipo. Indipendentemente dal fatto che contengano tipi per valore o tipi per riferimento, gli array sono strutturalmente considerati oggetti allocati sempre nello *Heap*.

3.5 Differenza tra tipi valore e tipi riferimento in .NET

In .NET, la distinzione tra tipi valore (*Value Types*) e tipi riferimento (*Reference Types*) determina come i dati vengono allocati in memoria, come vengono copiati e come avviene il ciclo di vita delle istanze:

- **Value types**: comprendono tutti i tipi numerici primitivi (`int`, `float`, `double`, `bool`, `char`), le strutture (`struct`) e le enumerazioni (`enum`). Le variabili di questo tipo memorizzano direttamente i dati effettivi all’interno del proprio spazio di memoria.
- **Reference types**: comprendono invece le classi (`class`), le interfacce (`interface`), i delegati (`delegate`), i tipi anonimi e il tipo speciale `string`. Le variabili di questo tipo non contengono i dati reali, bensì un indirizzo di memoria (puntatore) che fa riferimento alla posizione in cui i dati sono effettivamente allocati.

Le differenze fondamentali tra i due modelli si articolano sui seguenti aspetti ingegneristici:

- **Allocazione della memoria**: il CLR suddivide la memoria del processo in due aree logiche distinte, con algoritmi di gestione differenti. Lo Stack ospita i frame di attivazione dei metodi, incluse le variabili locali e i parametri. I tipi valore vengono allocati direttamente nello Stack quando dichiarati come variabili locali di un metodo. I tipi riferimento allocano la propria struttura dati (i campi dell’oggetto) e i metadati di runtime (*Method Table Pointer* e *SyncBlockIndex*) all’interno del Managed Heap, mentre lo Stack memorizza unicamente il riferimento (il puntatore da 32 o 64 bit) all’indirizzo nello Heap.
- **Semantica di copia e assegnamento**: regolata dall’operatore `=`, varia radicalmente in base alla natura del tipo. Nei tipi valore l’assegnamento esegue una copia integrale dell’intero contenuto della variabile: le due variabili diventano completamente indipendenti e risiedono in slot di memoria separati nello Stack, per cui una modifica sulla seconda non altera in alcun modo la prima. Nei tipi riferimento l’assegnamento duplica unicamente il riferimento (il puntatore memorizzato nello Stack), non l’oggetto reale nello Heap: di conseguenza, entrambe le variabili punteranno alla stessa identica istanza, e qualsiasi modifica apportata tramite una variabile si rifletterà istantaneamente sull’altra (*side-effect*).
- **Ciclo di vita e Garbage Collection**: i due comportamenti sono differenziati in modo netto. I tipi valore non gravano sul Garbage Collector: la loro de-allocazione è deterministica, immediata e automatica, e avviene non appena il thread di esecuzione esce dal contesto (*scope*) o dal metodo in cui la variabile è stata dichiarata, distruggendo il relativo frame dello Stack. Per i tipi riferimento, invece, il ciclo di vita è governato dal Garbage Collector in modalità non deterministica.
- **Gestione del valore nullo (null)**: i tipi valore sono obbligati a rappresentare sempre un valore valido coerente con il proprio dominio binario, motivo per cui non possono assumere nativamente il valore `null`. Al contrario, i tipi riferimento supportano nativamente il valore `null`, il quale indica un puntatore che non indirizza alcuna area di memoria valida dello Heap; un tentativo di accesso ai membri di un riferimento nullo solleva un’eccezione di tipo `NullReferenceException`.

3.6 Garbage Collector in C#

Il Garbage Collector (GC) è un componente del runtime di .NET preposto alla gestione automatica del ciclo di vita della memoria dinamica (Managed Heap). Il suo scopo primario è sollevare il programmatore dall'onere di dover allocare e de-allocare manualmente la memoria occupata dagli oggetti non più utilizzati dall'applicazione.

Nello sviluppo software tradizionale (es. C/C++), la gestione della memoria è manuale ed esplicita: ogni blocco di memoria allocato deve essere liberato tramite apposite istruzioni del programmatore. Nei linguaggi gestiti come C#, invece, il flusso viene invertito: il runtime alloca lo spazio nello Heap a fronte dell'istruzione `newobj`, mentre il Garbage Collector monitora in background lo stato di raggiungibilità delle istanze e provvede autonomamente alla loro rimozione quando queste non sono più referenziate da alcuna parte del programma.

L'adozione di un Garbage Collector permette di risolvere in modo sistematico tre problemi precisi intrinseci della gestione manuale della memoria:

1. **Memory Leaks (Perdite di memoria):** Si verificano quando la memoria per un oggetto viene allocata nello Heap ma il programmatore omette di liberarla al termine del suo utilizzo. Se tale comportamento si ripete all'interno di routine frequenti, la memoria occupata cresce progressivamente fino a saturare le risorse del sistema operativo, provocando il crash dell'applicazione per esaurimento della memoria (`OutOfMemoryException`). Il GC previene questa anomalia rilevando autonomamente gli oggetti orfani e reclamando il loro spazio.
2. **Dangling Pointers (Puntatori penzolanti):** Si generano quando la memoria di un oggetto viene de-allocata esplicitamente, ma nel programma permangono variabili o riferimenti attivi che puntano ancora a quel vecchio indirizzo di memoria. Un successivo tentativo di accesso tramite tali riferimenti causa comportamenti imprevisti, corruzione dei dati o violazioni di accesso. Il GC risolve il problema garantendo che un oggetto non venga mai rimosso finché esiste almeno un riferimento attivo ad esso.
3. **Double-Free (Doppia de-allocazione):** Accade quando il codice tenta di liberare due volte consecutive lo stesso blocco di memoria Heap. Questa operazione può corrompere le strutture dati interne del gestore di memoria del sistema operativo ed è storicamente una vulnerabilità sfruttata per eseguire codice maligno. Essendo il GC l'unico modulo autorizzato a de-allocare la memoria nel runtime, il rischio di doppie eliminazioni viene azzerato alla radice.

Tuttavia, i benefici della gestione automatica comportano un compromesso prestazionale (overhead): il GC richiede tempo di calcolo della CPU per l'analisi dei grafi, introduce un consumo addizionale di memoria per le strutture dati di tracciamento e determina un comportamento non deterministico, in quanto l'istante esatto di attivazione della pulizia non è prevedibile a priori e può indurre micro-pause nell'applicazione.

La gestione automatica della memoria può seguire tre principi teorici fondamentali:

- **Tracing:** Si mantiene traccia di tutti gli oggetti raggiungibili, eliminando quelli che non lo sono. Un oggetto è considerato raggiungibile solamente se viene riferito attraverso un oggetto raggiungibile (oppure è un oggetto radice). L'algoritmo interpreta la memoria come un grafo orientato in cui i nodi sono le allocazioni e gli archi sono i riferimenti. All'interno di questo modello si definiscono:
 - *Raggiungibilità Sintattica:* Oggetto che il programma non può raggiungere a causa dell'assenza di archi entranti a partire dal set delle radici stabili.
 - *Raggiungibilità Semantica:* Oggetto che il programma non vuole raggiungere; l'oggetto è staticamente raggiungibile nel grafo, ma il flusso logico del codice non eseguirà mai più alcuna operazione su di esso, configurando un'inefficienza di memoria.

- **Reference Counting:** Ad ogni oggetto viene associato un contatore di riferimenti ad esso. Questo approccio non tiene in considerazione le referenze circolari (isole di oggetti non raggiungibili dalle radici ma con conteggio maggiore di zero), aumenta l'uso di memoria per l'allineamento dei contatori e necessita di atomicità hardware (istruzioni di tipo Interlocked) per le operazioni di incremento/decremento in ambienti multithread.
- **Escape Analysis:** Analisi compile-time per stabilire se un oggetto è necessario all'esterno di una sub-routine. Attraverso il calcolo della visibilità del riferimento, se l'oggetto non "evade" dal record di attivazione locale, il compilatore ne dispone l'allocazione diretta sullo Stack, azzerando l'overhead di garbage collection.

In .NET viene utilizzata la tecnica del tracing. In fase di inizializzazione di un processo, il CLR riserva una regione contigua di spazio detta managed heap e memorizza il puntatore `NextObjPtr` all'indirizzo di partenza di tale area. All'esecuzione di `newobj`, il runtime calcola la dimensione in byte dell'oggetto aggiungendo due campi overhead da 32 bit (su architetture x86) o 64 bit (su architetture x64), corrispondenti al puntatore alla tabella dei metodi (*Method Table Pointer*) e al `SyncBlockIndex`. Successivamente controlla che ci sia spazio sufficiente a partire da `NextObjPtr` per allocare l'oggetto. Se non fosse possibile, viene eseguito il garbage collector o, in caso di fallimento definitivo, viene lanciata un'eccezione di tipo `OutOfMemoryException`. A questo punto il puntatore all'oggetto diventa `NextObjPtr`, il quale viene infine incrementato della dimensione calcolata.

Nel dettaglio, le fasi del garbage collector sono:

- **Mark:** Nel momento di esecuzione del garbage collector, questo marca tutti gli oggetti come non raggiungibili azzerando il bit di validità nell'header. Partendo dalle radici (*GC Roots*, ovvero variabili globali, statiche, variabili locali o argomenti memorizzati nei frame attivi dello Stack, e registri della CPU caricati dal compilatore JIT) marca a cascata tutti gli oggetti vivi, ovvero referenziati. Salta tutti gli oggetti già marcati per interrompere i cicli infiniti e ottimizzare i tempi di scansione. Questa fase viene eseguita in modalità "Stop-The-World" per congelare lo stato del sistema ed evitare inconsistenze.
- **Compact:** Rilascia la memoria non utilizzata e ricompatta gli oggetti superstiti, spostandoli fisicamente per renderli contigui ed evitando così la frammentazione interna ed esterna dello heap. Questo è possibile perché .NET è un ambiente fortemente tipizzato (*type-safe*): il tipo di dato è noto ed è possibile utilizzare i metadati del runtime per determinare quali field puntano verso altri oggetti; il GC aggiorna quindi chirurgicamente i vecchi indirizzi di memoria contenuti nei riferimenti dello Stack e degli altri oggetti rilocati.

Prima della pulizia dello heap gestito, rimane comunque compito del programmatore liberare tempestivamente tutte quelle risorse unmanaged (file, database, socket di rete, ecc.). Poiché il Garbage Collector mappa unicamente la memoria gestita e ignora i descrittori del sistema operativo, il rilascio di tali risorse deve essere implementato esplicitamente tramite l'interfaccia `IDisposable` (metodo `Dispose()`) o affidato in modo non deterministico ai Finalizer (metodi distruttori) invocati da un apposito thread del runtime prima della de-allocazione fisica del blocco di memoria.

3.7 Ciclo di vita di un oggetto esemplificandone i passi

Il ciclo di vita di un oggetto è composto da quattro fasi:

1. **Allocazione della memoria:** avviene tramite l'operatore `new` (*alloc*), che riserva lo spazio necessario per l'oggetto sul Managed Heap (per i tipi riferimento) o sullo Stack (per i tipi valore). Ad esempio, l'istruzione `Persona p = new Persona();` riserva sullo Heap lo spazio per i campi dell'oggetto `Persona`, mentre sullo Stack viene riservato solo lo spazio per il riferimento `p`. In questa fase il runtime azzerla la memoria riservata, garantendo che tutti i campi dell'oggetto partano da uno stato noto (default) prima ancora che venga eseguito il costruttore.

2. **Inizializzazione:** è conseguenza dell'esecuzione del costruttore (*ctor*) e, eventualmente, di metodi di istanza invocati durante la costruzione. Il compilatore garantisce che ad ogni variabile venga assegnato un valore prima del suo utilizzo: a tale scopo, possono intervenire valori di default oppure costruttori espliciti definiti dal programmatore. Ad esempio, nel costruttore:

```
Persona(string nome) { Nome = nome; }
```

il campo `Nome` riceve il valore passato come parametro, mentre eventuali altri campi non inizializzati esplicitamente assumono il proprio valore di default (`0`, `null`, `false`, ecc.). I costruttori possono inoltre essere concatenati tramite le keyword `this` (per richiamare un altro costruttore della stessa classe) o `base` (per richiamare il costruttore della classe padre), in modo da evitare duplicazione di codice di inizializzazione.

3. **Rilascio delle risorse esterne:** avviene tramite il distruttore (C#/C++), che è unico per classe e non ereditabile, oppure tramite il metodo `Finalize` (Java), invocato dal Garbage Collector in modo non deterministico. In C# la sintassi del distruttore (`~NomeClasse`) viene tradotta dal compilatore in un override del metodo `Finalize`: il programmatore non può quindi invocarlo né prevederne con certezza il momento di esecuzione. Per le risorse che richiedono un rilascio deterministico e immediato (file, connessioni, handle di sistema), la convenzione .NET prevede l'implementazione dell'interfaccia `IDisposable` e del metodo `Dispose`, tipicamente invocato tramite il costrutto `using`. Ad esempio:

```
using (var reader = new StreamReader(path)) {...}
```

garantisce che `Dispose` venga chiamato sull'oggetto `reader` non appena l'esecuzione esce dal blocco, anche in presenza di un'eccezione.

4. **Liberazione della memoria:** per i tipi valore dichiarati come variabili locali, la memoria viene liberata automaticamente all'uscita dallo scope (*out of scope*), mediante la semplice distruzione del frame dello Stack: ad esempio, una variabile `int x` dichiarata all'interno di un metodo cessa di esistere non appena il metodo termina. Questo, tuttavia, non vale in generale: se un tipo valore è un campo di un oggetto allocato sullo Heap (ad es. il campo `int Età` all'interno di un oggetto `Persona`), la sua memoria viene liberata insieme a quella dell'oggetto contenitore, e quindi segue il ciclo di vita del tipo riferimento. Per i tipi riferimento, infatti, è sempre il Garbage Collector a recuperare (*reclaim*) lo spazio occupato nello Heap, operando in modo non deterministico: l'oggetto `Persona` creato in precedenza, una volta che nessuna variabile lo riferisce più (ad es. dopo l'uscita dallo scope di `p`, o dopo `p = null;`), diventa *eleggibile* per la raccolta, ma la sua effettiva deallocazione avverrà solo al successivo ciclo del GC, e non è garantito che avvenga in un momento prevedibile.

3.8 Passaggio dei parametri in C#

In C# il passaggio dei parametri può essere eseguito secondo tre modalità differenti, ciascuna delle quali impone specifici vincoli di inizializzazione e definisce la natura del valore allocato sullo Stack:

- **In (default):** L'argomento deve essere obbligatoriamente inizializzato prima della chiamata e viene passato per valore. Il metodo chiamato riceve una copia locale del parametro nello Stack; pertanto, eventuali modifiche effettuate all'interno del metodo non si riflettono sulla variabile originale del chiamante.
- **In/Out (ref):** L'argomento deve essere obbligatoriamente inizializzato prima della chiamata e viene passato per riferimento tramite la parola chiave `ref`. Il metodo riceve l'indirizzo di memoria della variabile originale; pertanto, le modifiche si riflettono direttamente sull'originale.

- **Out (out):** L'argomento può non essere inizializzato prima della chiamata e viene passato per riferimento tramite la parola chiave **out**. Le modifiche si riflettono sull'originale, in quanto viene passato il suo indirizzo di memoria. L'argomento ha il vincolo tassativo di essere inizializzato all'interno del metodo chiamato prima di qualsiasi lettura o della sua terminazione.

Il passaggio **in** può quindi essere:

- **Value type:** Viene passata una copia esatta del valore dell'oggetto. La variabile del chiamante e il parametro del metodo occupano due slot di memoria distinti sullo Stack. Qualsiasi modifica isola il comportamento del metodo.
- **Reference type:** Viene passata una copia del riferimento (puntatore) all'oggetto. Sullo Stack viene duplicato solo il puntatore, ma entrambi i puntatori (quello del chiamante e quello del metodo) indirizzano la stessa identica area di memoria allocata nello Heap. Di conseguenza, le modifiche allo stato interno dell'oggetto (*side-effects*) sono visibili al chiamante, mentre riassegnare il parametro a un nuovo oggetto (**new**) altera solo la copia locale del puntatore, lasciando invariato l'originale.

Il passaggio **ref** può quindi essere:

- **Value type:** Viene passato direttamente l'indirizzo di memoria dell'oggetto allocato sullo Stack del chiamante. Non avviene alcuna duplicazione di dati; il metodo opera direttamente sulla locazione di memoria originale.
- **Reference type:** Viene passato l'indirizzo del riferimento all'oggetto (un puntatore a un puntatore). Se il metodo chiamato riassegna il parametro istanziando un nuovo oggetto tramite la parola chiave **new**, l'indirizzo memorizzato nella variabile originale del chiamante viene modificato, puntando stabilmente alla nuova area dello Heap.

Il passaggio **out** può quindi essere:

- **Value type:** Viene passato l'indirizzo di memoria dell'oggetto sullo Stack. Dal punto di vista del layout di memoria il comportamento è identico a **ref**, ma il compilatore consente il passaggio di variabili non inizializzate.
- **Reference type:** Viene passato l'indirizzo del riferimento all'oggetto nello Stack. Come per i primi, la variabile può non essere inizializzata nel chiamante.

L'oggetto o il riferimento devono essere inizializzati nel metodo nel quale sono stati passati come argomento.

3.9 Differenze tra interfaccia e classe astratta

La distinzione tra interfaccia e classe astratta rappresenta uno dei punti cardine della progettazione orientata agli oggetti e risponde a due necessità architetturali differenti, in particolare:

- **Interfaccia (Interface):** Rappresenta un tipo puro di specifica. È un contratto formale che definisce esclusivamente una collezione di firme di metodi (operazioni pubbliche), senza fornire alcuna implementazione del comportamento e senza definire uno stato (attributi d'istanza). Nei moderni linguaggi orientati agli oggetti è possibile definire metodi di *default* (dotati di una implementazione standard) e metodi statici. L'interfaccia deve essere stabile nel tempo in accordo con l'Open-Closed Principle (OCP), riducendo l'impatto di regressione sui moduli clienti. Dal punto di vista strutturale, può essere implementata più volte da una singola classe e può estendere a sua volta altre interfacce.

- **Classe Astratta (Abstract Class):** È una classe parzialmente implementata, contrassegnata dalla parola chiave **abstract**, che non può essere istanziata direttamente. Può contenere sia metodi astratti (privi di corpo, che fungono da specifica per le sottoclassi), sia metodi concreti (che forniscono un'implementazione comune riusabile). A differenza dell'interfaccia, può definire uno stato interno completo attraverso attributi privati, protetti o pubblici e possedere dei costruttori. Dal punto di vista strutturale, una classe può estendere un'unica classe (astratta o concreta) a causa del vincolo dell'ereditarietà semplice, ma tale classe astratta può estendere un'altra classe e implementare contemporaneamente più interfacce.

In fase di progettazione, la scelta tra i due costrutti è guidata dalla natura della relazione semantica che si intende instaurare:

- La classe astratta definisce una relazione di tipo **Is-A**. Esprime un legame di forte appartenenza concettuale e tassonomica a una famiglia di oggetti (es. **Cane Is-A Animale**), condividendone lo stato comune.
- L'interfaccia definisce una relazione di tipo **Can-Do** o **Has-A-Behavior**. Identifica una funzionalità, una capacità o un protocollo d'azione ortogonale rispetto alla gerarchia principale delle classi (es. sia **Aereo** che **Aquila** possono implementare l'interfaccia **IVolante**, pur appartenendo a domini concettuali disgiunti).

3.10 Concetto di delegato in C#

In C# il delegato è un tipo di dato sicuro (*type-safe*) che mantiene il riferimento a uno o più metodi e che ne permette l'invocazione differita nel tempo. È concettualmente equivalente ad un puntatore a funzione del C/C++, ma a differenza di quest'ultimo racchiude un approccio rigorosamente *object-oriented*, incapsulando contemporaneamente la firma del metodo e l'istanza dell'oggetto target. Questo costrutto viene utilizzato principalmente per supportare tre scenari architetturali:

- **Elaborazione asincrona** (attraverso l'esecuzione di callback su thread differenti).
- **Elaborazione cooperativa** (consentendo a moduli software distinti di scambiarsi comportamenti dinamici a runtime).
- **Gestione degli eventi** (attuando il pattern *Publisher-Subscriber* nativo del framework .NET).

È possibile assegnare ad un delegato una lista di metodi chiamata *invocation list*; in questo scenario il delegato viene definito *multicast delegate*. I metodi registrati vengono chiamati in sequenza ordinata ed in modo sincrono rispetto all'invocazione del delegato stesso. Ogni entità contenuta in un delegate è detta *callable entity* e, in caso di metodi di istanza, comprende sia l'indirizzo del codice del metodo sia l'istanza specifica dell'oggetto sulla quale il metodo deve essere invocato (memorizzata nella proprietà interna **Target**). Se il metodo associato è statico, la proprietà **Target** assume il valore **null**.

Il delegato applica e vincola una singola firma del metodo (definita in fase di dichiarazione del tipo tramite tipo di ritorno e parametri) e passa a ciascun metodo della lista lo stesso identico insieme di parametri in ingresso. Nel caso in cui la firma preveda dei valori di ritorno, l'invocazione sequenziale della lista sovrascriverà progressivamente i risultati parziali; di conseguenza, il valore finale restituito al chiamante sarà esclusivamente il risultato dell'invocazione dell'ultimo delegato presente nella lista.

3.11 Concetto di evento in C#

Un evento in C# è la formalizzazione sintattica di un avvenimento significativo all'interno del sistema, strutturata per implementare nativamente il pattern architetturale *Publisher-Subscriber*. Questa architettura si suddivide in tre componenti logiche distinte:

- **Event Sender (Publisher):** l'oggetto sorgente che detiene la logica di stato e che scatena (*fires*) l'evento quando si verifica la condizione predefinita.
- **Event Receiver (Subscriber):** l'oggetto destinatario che desidera essere notificato del cambiamento di stato avvenuto nel sender.
- **Event Handler:** il metodo di callback, definito nel receiver, la cui firma corrisponde a quella del delegato associato all'evento e che viene eseguito al momento della notifica.

L'event sender notifica il messaggio a tutti i receiver registrati per mezzo di un delegato sottostante, poiché non conosce singolarmente né gli handler né i receiver. Un evento, infatti, incapsula un delegato, e la convenzione di progettazione .NET impone che la sua firma preveda due parametri:

- la sorgente che ha generato l'evento (`object sender`);
- i dati associati all'evento (`EventArgs`), che possono essere omessi oppure estesi tramite sottoclassi personalizzate.

Per dichiarare un evento è quindi necessario definire la firma del delegato di riferimento e successivamente applicare la keyword `event` davanti alla dichiarazione del campo, specificando come tipo il delegato stesso. La parola chiave `event` agisce come un modificatore di visibilità e di incapsulamento, poiché limita i diritti di accesso al delegato sottostante. Esternamente alla classe che lo dichiara, le sole operazioni permesse sull'evento sono:

- l'aggancio (`+=`) di un event handler;
- lo sganciamento (`-=`) di un event handler.

Nel processo di pubblicazione di un evento, il runtime deve verificare se il delegato incapsulato sia uguale a `null`, condizione che si verifica qualora nessun receiver si sia ancora registrato. Per scatenare l'evento in modo sicuro, si definisce convenzionalmente un metodo protetto e virtuale denominato `OnNomeEvento`, il quale esegue il controllo di validità e invoca il delegato. L'invocazione diretta dell'evento (e di conseguenza dell'*invocation list* del delegato) può avvenire esclusivamente all'interno della classe che dichiara l'evento stesso, impedendo a entità esterne di violare l'incapsulamento della sorgente.

3.12 Metaprogrammazione e riflessione in C#

Con riflessione si intende la capacità di un sistema di leggere le informazioni su se stesso in fase di runtime, ossia di ispezionare a runtime la struttura di assembly, moduli e tipi (classi, interfacce, metodi, proprietà, campi, attributi) senza conoscerla a tempo di compilazione. La capacità di modificare tali informazioni è invece detta metaprogrammazione.

La riflessione sfrutta i metadati ed è implementata in C# attraverso il namespace `System.Reflection`. Diversamente da tecnologie come COM e CORBA, in cui i metadati venivano descritti separatamente tramite un IDL (*Interface Definition Language*) e distribuiti in file a parte, in .NET i metadati sono generati

direttamente dal compilatore a partire dal codice sorgente e vengono memorizzati insieme ad esso all'interno dell'assembly, sotto forma di tabelle (tipi, membri, parametri, attributi personalizzati, ecc.).

La reflection in C# si basa sulla classe `System.Type`, che costituisce il punto di ingresso per l'introspezione: ogni tipo all'interno del sistema è rappresentato da un'istanza di `Type`, e per ciascun tipo definito nel sistema esiste un singolo oggetto `Type` associato (ottenibile tramite l'operatore `typeof`, oppure tramite il metodo `GetType()` invocato su un'istanza). A partire da un oggetto `Type` è possibile navigare l'intera struttura del tipo, ottenendo oggetti come `MethodInfo`, `PropertyInfo`, `FieldInfo` e `ConstructorInfo`, che descrivono rispettivamente metodi, proprietà, campi e costruttori.

Tali tecnologie permettono l'utilizzo di metodi come:

- **Assembly.Load**: carica un assembly in memoria, a partire dal suo nome o dal percorso del file, rendendone disponibili i metadati per l'ispezione.
- **Assembly.GetModules**: restituisce i moduli che compongono l'assembly (un assembly può infatti essere composto da più moduli).
- **Module.GetTypes**: restituisce tutti i tipi definiti all'interno di un modulo, consentendo di enumerare dinamicamente le classi disponibili.

oltre che operazioni di *late binding*, ossia la possibilità di scegliere a runtime quale tipo, metodo o membro invocare, tramite:

- **Activator.CreateInstance**: crea dinamicamente un'istanza di un tipo, anche quando il tipo non è noto a tempo di compilazione (ad es. caricato da un assembly esterno).
- **MethodInfo.Invoke**: invoca dinamicamente un metodo su un'istanza (o staticamente, se il metodo è `static`), passando i parametri come array di `object`.
- **propertyInfo.GetValue**: legge dinamicamente il valore di una proprietà su una data istanza.
- **propertyInfo.SetValue**: imposta dinamicamente il valore di una proprietà su una data istanza.

La metaprogrammazione viene invece implementata attraverso:

- **System.Reflection**: oltre all'ispezione, consente la creazione di delegati dinamici e l'accesso a metadati personalizzati tramite gli *attributi* (`Attribute`), che possono essere applicati a classi, metodi o proprietà e letti a runtime per modificarne il comportamento.
- **System.Reflection.Emit**: permette di generare dinamicamente, a runtime, nuovo codice IL (nuovi assembly, tipi, metodi), che viene poi eseguito dal CLR esattamente come codice compilato in modo statico.

4 Design Principle

4.1 Spiegare i quattro bad design (fragilità, immobilità, rigidità, viscosità)

Le quattro principali manifestazioni di un *bad design* sono:

- **Fragilità**: tendenza del software a rompersi in punti inaspettati a seguito di una singola modifica, anche in parti del sistema concettualmente non correlate a quella modificata.

- **Immobilità:** il sistema è composto da moduli estremamente interdipendenti e quindi difficilmente riutilizzabili. Nel caso di un tentativo di riutilizzo, sarebbe necessario un lavoro di separazione dei moduli talmente oneroso da spingere gli sviluppatori a riscrivere da zero le funzionalità già presenti, piuttosto che riutilizzarle.
- **Rigidità:** tendenza del software ad essere difficilmente modificabile. Si parla di rigidità quando una singola modifica provoca una cascata di modifiche successive nei moduli dipendenti: ogni correzione, oltre a richiedere ulteriori interventi a catena, rischia di introdurre nuovi problemi, peggiorando progressivamente lo stato del software.
- **Viscosità:** tendenza di un software a favorire modifiche di tipo "hack", che non rispettano l'intento di progettazione originale, piuttosto che modifiche corrette ma più laboriose. Questo può avere due cause:
 - **Viscosità del design:** i metodi "hack", che violano il design originale, risultano più semplici e veloci da applicare rispetto ai metodi che preservano il design.
 - **Viscosità dell'ambiente:** l'ambiente di sviluppo (build, test, deploy) è lento e inefficiente, per cui gli sviluppatori sono incentivati a evitare modifiche "corrette" ma che richiederebbero cicli di verifica troppo lunghi.

Le cause che portano allo sviluppo di tali problemi possono essere sia ovvie (mancanza di competenze, tecnologie in evoluzione, vincoli di tempo, ecc.) sia meno ovvie, come:

- putrefazione del software (*software rot*): degrado progressivo della struttura del codice causato dall'accumularsi di modifiche non pianificate;
- requisiti variabili: cambiamenti nei requisiti, spesso non previsti nel design originale, che costringono ad adattamenti forzati;
- dipendenze non pianificate: relazioni tra moduli che emergono in modo non intenzionale durante lo sviluppo, non previste nella fase di progettazione iniziale.

4.2 Principio di singola responsabilità con almeno un esempio

Secondo il principio di singola responsabilità, una classe deve possedere sempre e solo una responsabilità, intesa come un unico motivo per cui potrebbe essere modificata. La classe si occupa di tale responsabilità nel modo corretto e in maniera esclusiva.

Una classe che accorpa più responsabilità può portare, nel momento in cui una di esse necessita di una modifica, a compromettere il corretto funzionamento delle altre, oppure a rendere la classe difficilmente estendibile per l'aggiunta di ulteriori responsabilità. Questo porta a design fragili, in cui una modifica apparentemente isolata può propagarsi in modo inaspettato.

Si consideri ad esempio una classe `GestoreCambioPassword` che, oltre a validare e aggiornare la password dell'utente, si occupa anche di salvare i nuovi dati sul database e di inviare un'email di conferma dell'avvenuto cambio:

- una modifica alla logica di validazione della password (ad es. nuovi requisiti di complessità) richiede di modificare la classe `GestoreCambioPassword`;
- una modifica alla modalità di accesso al database (ad es. il passaggio a un nuovo ORM o a un diverso DBMS) richiede anch'essa di modificare la stessa classe;
- una modifica al contenuto o al provider del servizio di invio email (ad es. il passaggio a un nuovo servizio SMTP, o una nuova grafica del messaggio) richiede ancora una volta di modificare `GestoreCambioPassword`.

La classe avrebbe quindi tre motivi distinti per cambiare, violando l'SRP: una modifica al servizio di invio email, ad esempio, rischia di introdurre un errore nella logica di salvataggio su database (o viceversa), pur trattandosi di responsabilità concettualmente indipendenti. Inoltre, testare la sola logica di validazione della password diventa difficile, poiché ogni test coinvolgerebbe inevitabilmente anche il database e il servizio email.

La soluzione consiste nel suddividere la classe in più classi, ciascuna con una singola responsabilità: ad esempio `ValidatorePassword` (validazione della nuova password secondo le policy di sicurezza), `RepositoryUtente` (persistenza dei dati sul database) e `NotificatoreEmail` (invio dell'email di conferma). La classe `GestoreCambioPassword` si limita a orchestrare le tre operazioni, delegando ciascuna alla classe competente: in questo modo, una modifica al provider email coinvolge esclusivamente `NotificatoreEmail`, senza alcun impatto sulle altre classi né sulla logica di validazione.

4.3 Principio di inversione delle dipendenze con almeno un esempio

Secondo il principio di inversione delle dipendenze, ogni dipendenza dovrebbe puntare a un'interfaccia o a una classe astratta, piuttosto che a una classe concreta. I moduli di alto livello, in particolare, non dovrebbero dipendere dai moduli di basso livello: entrambi dovrebbero invece dipendere da astrazioni.

Questo principio nasce dal fatto che i moduli di basso livello (ad es. dettagli di implementazione, accesso a risorse esterne) hanno una maggiore probabilità di essere soggetti a cambiamenti rispetto ai moduli di alto livello (che incapsulano la logica di business). In presenza di una dipendenza diretta tra alto e basso livello, tali cambiamenti si propagherebbero verso l'alto, generando i problemi di fragilità, rigidità e immobilità. Le astrazioni, al contrario, non dovrebbero contenere logica implementativa e sono quindi intrinsecamente più stabili nel tempo. Questo rende inoltre i moduli più riutilizzabili, poiché disaccoppiati dai dettagli implementativi specifici.

Si consideri ad esempio un modulo di alto livello `GestoreOrdini`, che deve notificare l'avvenuta evasione di un ordine. Una progettazione che viola il DIP farebbe dipendere `GestoreOrdini` direttamente da una classe concreta di basso livello, ad esempio `NotificatoreEmail`: in tal caso, ogni cambiamento nel canale di notifica (ad es. l'introduzione di notifiche via SMS o push, oppure il cambio del provider email) richiederebbe di modificare anche `GestoreOrdini`, oltre che impedirne il test in isolamento senza inviare email reali.

Applicando il DIP, si introduce un'astrazione `INotificatore`, da cui dipende `GestoreOrdini`, e che viene implementata dalle classi concrete `NotificatoreEmail`, `NotificatoreSMS`, ecc. In questo modo:

- il modulo di alto livello (`GestoreOrdini`) dipende solo dall'astrazione `INotificatore`, e non dai dettagli implementativi;
- i moduli di basso livello (`NotificatoreEmail`, `NotificatoreSMS`) dipendono a loro volta dalla stessa astrazione, implementandola;
- è possibile aggiungere nuovi canali di notifica, o sostituire quelli esistenti, senza modificare `GestoreOrdini`, e testare `GestoreOrdini` fornendo un'implementazione fittizia (*mock*) di `INotificatore`.

I tipi di dipendenze tra moduli sono:

- **dipendenze transitive:** si verificano quando un modulo dipende indirettamente da un altro attraverso una catena di dipendenze intermedie (A dipende da B, che dipende da C, quindi A dipende transitivamente da C);
- **dipendenze cicliche:** si verificano quando due o più moduli dipendono l'uno dall'altro, direttamente o tramite una catena di dipendenze, formando un ciclo. Le dipendenze cicliche sono particolarmente problematiche perché rendono i moduli inseparabili e non testabili/compilabili in isolamento.

È altresì importante che le astrazioni in uso non vengano modificate, ma estese: modificare un'astrazione esistente avrebbe infatti un impatto su tutti i moduli che ne dipendono, vanificando i benefici del disaccoppiamento.

4.4 Principio di segregazione delle interfacce con almeno un esempio

Secondo il principio di segregazione delle interfacce, un client non dovrebbe essere costretto a dipendere da metodi di un'interfaccia di cui non ha bisogno. Si preferisce quindi avere un numero maggiore di interfacce piccole e specifiche (*role interfaces*), ciascuna pensata per un particolare client, piuttosto che un'unica interfaccia "fat" contenente tutti i metodi: quest'ultima creerebbe infatti un accoppiamento superfluo tra client che, in realtà, hanno esigenze completamente diverse.

Si consideri ad esempio un sistema di gestione utenti che espone un'unica interfaccia `IGestioneUtente`, contenente sia i metodi destinati agli utenti normali sia quelli riservati agli amministratori.

Una classe `PannelloUtente`, che rappresenta l'interfaccia utilizzata da un utente normale, sarebbe costretta a dipendere dall'intera interfaccia `IGestioneUtente`, anche se utilizza solo i metodi di un utente normale. Questo comporta diversi problemi:

- `PannelloUtente` dipende (e potrebbe essere ricompilato o influenzato) anche da modifiche ai metodi amministrativi, che non utilizza;
- Un'implementazione fittizia (*mock*) di `IGestioneUtente` per testare `PannelloUtente` dovrebbe comunque fornire un'implementazione, anche vuota, dei metodi amministrativi.

Applicando l'ISP, l'interfaccia viene segregata in due interfacce distinte: `IProfiloUtente` e `IAmministrazioneUtente`. La classe concreta che gestisce gli utenti (ad es. `GestoreUtente`) può implementare entrambe le interfacce, ma:

- `PannelloUtente` dipenderà solo da `IProfiloUtente`, ignorando completamente l'esistenza dei metodi amministrativi;
- `PannelloAdmin` dipenderà invece da `IAmministrazioneUtente`.

In questo modo, una modifica ai metodi amministrativi non ha alcun impatto su `PannelloUtente`, che continua a dipendere unicamente da `IProfiloUtente`.

4.5 Principio aperto/chiuso con almeno un esempio

Secondo il principio aperto/chiuso, le entità software (classi, moduli, funzioni) dovrebbero essere aperte all'estensione ma chiuse alla modifica. Ciò significa che deve essere possibile estenderne il comportamento, ad esempio aggiungendo nuovo stato o nuove funzionalità, senza dover modificare il codice sorgente esistente. A tale scopo, le entità devono esporre un'interfaccia pubblica ben definita e stabile, che non viene mai alterata. È quindi necessario progettare moduli estendibili che non richiedano cambiamenti al proprio interno per accogliere nuovi requisiti: questo si ottiene tipicamente attraverso astrazioni stabili (interfacce o classi astratte), che vengono poi estese dalle singole implementazioni concrete.

Si consideri ad esempio una classe `CalcolatoreSconto` che calcola lo sconto da applicare a un ordine in base al tipo di cliente attraverso `double CalcolaSconto(Ordine ordine)`.

Questo viola l'OCP poiché ogni volta che viene introdotta una nuova tipologia di cliente (ad es. "Affiliato"), è necessario modificare il metodo `CalcolaSconto`, aggiungendo un nuovo ramo `else if`.

Applicando l'OCP, si introduce un'astrazione `IStrategiaSconto`, con un unico metodo `Calcola(Ordine ordine)`, implementata da una classe concreta per ciascuna tipologia di cliente (`ScontoStandard`, `ScontoPremium`, `ScontoVIP`). La classe `CalcolatoreSconto` dipende unicamente dall'astrazione `IStrategiaSconto`, ricevendo l'implementazione concreta opportuna. In questo modo:

- l'interfaccia `IStrategiaSconto` e la classe `CalcolatoreSconto` restano chiuse alla modifica: il loro codice non viene mai alterato;
- il sistema resta aperto all'estensione: per supportare la nuova tipologia "Affiliato" è sufficiente aggiungere una nuova classe `ScontoAffiliato` che implementa `IStrategiaSconto`, senza toccare il codice esistente né rischiare di introdurre regressioni nelle logiche di sconto già testate.

4.6 Principio di sostituibilità di Liskov con almeno un esempio

Secondo il principio di sostituibilità di Liskov, ogni sottoclasse dovrebbe essere in grado di sostituire la propria classe base senza alterare il comportamento atteso del sistema. Ogni client al quale viene passata un'istanza di una sottoclasse deve continuare a funzionare correttamente, come se stesse lavorando con un'istanza della classe base, senza accorgersi della differenza.

Questo principio rappresenta una guida per l'applicazione dell'OCP: è infatti l'LSP che garantisce che l'estensione di una classe base tramite nuove sottoclassi non comprometta la correttezza del codice client esistente. La violazione di questo principio deriva spesso dalla ridefinizione di metodi virtuali (*override*) che alterano in modo incompatibile il comportamento atteso dalla classe base.

Al fine di non violare il principio, si fa riferimento al *design by contract*. Ogni metodo è caratterizzato da una preconditione e da una postcondizione, che costituiscono un contratto tra il client e il metodo stesso: la preconditione è ciò che il metodo richiede per poter essere eseguito correttamente, mentre la postcondizione è ciò che il metodo garantisce al termine della propria esecuzione. Quando un metodo viene ridefinito in una sottoclasse, le preconditioni devono essere uguali o meno stringenti, mentre le postcondizioni devono essere identiche o più stringenti.

Si consideri ad esempio una classe base `ContoBancario`, con un metodo `Preleva(double importo)` la cui preconditione richiede semplicemente che `importo` sia positivo e non superiore al saldo disponibile, e la cui postcondizione garantisce che il prelievo venga sempre eseguito (il saldo viene decrementato dell'importo richiesto). Si supponga ora una sottoclasse `ContoRisparmio` che ridefinisce `Preleva` aggiungendo un vincolo: il prelievo è consentito solo se l'importo non supera una soglia massima mensile, altrimenti il metodo lancia un'eccezione.

Un client che riceve un `ContoBancario` (in realtà un'istanza di `ContoRisparmio`) e invoca `Preleva(importo)` con un `importo` positivo e non superiore al saldo, soddisfacendo quindi la preconditione prevista dalla classe base, si aspetta che il prelievo venga eseguito (postcondizione della classe base). La sottoclasse `ContoRisparmio`, però, introduce un'eccezione in più casi rispetto alla classe base: la sua preconditione effettiva è quindi più stringente (richiede anche il rispetto della soglia mensile), violando l'LSP. Il client, che non ha alcuna conoscenza della soglia mensile (introdotta solo nella sottoclasse), si troverebbe a gestire un'eccezione inattesa.

La soluzione consiste nell'esplicitare il vincolo sulla soglia già a livello dell'astrazione comune, ad esempio definendo nella classe base `ContoBancario` un metodo `Preleva(double importo)` la cui preconditione include, in generale, "il rispetto di eventuali limiti di prelievo definiti dal conto" (con limite predefinito pari a infinito, o assente, per i conti senza restrizioni). In questo modo, ogni sottoclasse (incluso `ContoRisparmio`) può specializzare il proprio limite senza introdurre preconditioni più stringenti di quelle dichiarate dalla classe base, e il client può sempre verificare a priori, tramite l'astrazione, se un prelievo è consentito.

4.7 Principi per l'architettura dei package

I principi di coesione tra i package sono tre:

- **Reuse/Release Equivalency Principle (REP):** un elemento riutilizzabile non può essere effettivamente riutilizzato se non è gestito da un sistema di rilascio, che assegni a ciascuna versione un numero identificativo e una documentazione delle modifiche. I clienti dovrebbero rifiutare l'utilizzo di package senza un sistema di tracciamento delle versioni.
- **Common Closure Principle (CCP):** le classi che cambiano per gli stessi motivi e nello stesso momento dovrebbero appartenere allo stesso package, mentre classi che cambiano per motivi diversi dovrebbero essere separate in package differenti. Più package vengono modificati durante un rilascio, infatti, maggiore sarà il lavoro di ricostruzione, testing e distribuzione: si vuole quindi ridurre al minimo il numero di package modificati durante un singolo ciclo di rilascio.
- **Common Reuse Principle (CRP):** classi non riutilizzate insieme non dovrebbero essere raggruppate nello stesso package, poiché un client che dipende da un package per riutilizzarne una sola classe si troverebbe comunque a dipendere (ed essere impattato da modifiche) anche dalle altre classi del package, anche se non le utilizza.

Questi tre principi non possono essere soddisfatti contemporaneamente: REP e CRP semplificano la vita ai clienti dei package (favorendo package piccoli, coesi e facilmente riutilizzabili), mentre CCP semplifica la vita ai manutentori (favorendo package grandi, che raggruppano tutto ciò che cambia insieme). In particolare, il CCP tende a creare package quanto più grandi possibile, mentre il CRP tende a creare package quanto più piccoli possibile: la progettazione dei package richiede quindi un compromesso tra queste forze contrapposte, in base al contesto e alla fase di vita del progetto.

È poi possibile descrivere le relazioni di dipendenza tra i package tramite altri tre principi:

- **Acyclic Dependencies Principle (ADP):** le dipendenze tra package non devono essere cicliche. La presenza di cicli rende complesso il testing e la compilazione in isolamento dei singoli package, poiché ciascuno di essi dipende, indirettamente, anche da se stesso. Per rompere un ciclo è possibile inframmezzare un nuovo package (in cui collocare gli elementi che causano il ciclo), oppure introdurre un'interfaccia che inverta la direzione di una delle dipendenze.
- **Stable Dependencies Principle (SDP):** un package dovrebbe dipendere solamente da package più stabili di lui (cioè meno soggetti a modifiche), in modo che le modifiche più frequenti siano confinate ai package meno stabili, senza propagarsi verso quelli più stabili.
- **Stable Abstractions Principle (SAP):** i package stabili dovrebbero essere anche astratti. I package altamente stabili, posizionati nella parte inferiore del grafo delle dipendenze, possono infatti risultare molto difficili da modificare; tuttavia, secondo l'OCP, non devono per questo essere difficili da *estendere*: rendendoli astratti, è possibile estenderne il comportamento tramite nuove implementazioni concrete, senza modificarne il codice.

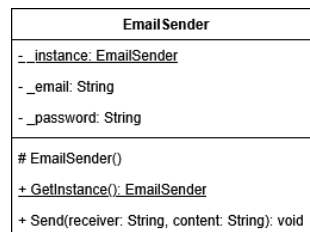
5 Design patterns

5.1 Pattern Singleton con esempi

Il pattern Singleton è un pattern creazionale che assicura che possa esistere una sola istanza di una classe all'interno dell'intero ciclo di vita di un'applicazione, fornendo al contempo un punto di accesso globale e controllato a tale istanza.

Una possibile alternativa sintattica per accentrare funzionalità globali è l'utilizzo di una classe non istanziabile contenente esclusivamente metodi statici. Tuttavia, ogni Singleton può implementare più interfacce ed essere specializzato tramite ereditarietà, permettendo al metodo `GetInstance` di variare senza che il chiamante debba conoscerne i dettagli implementativi.

Un esempio pratico di Singleton è dato dalla classe manager dedicata all'invio delle email all'interno di un sistema. Questa entità centrale si occupa della gestione centralizzata di tutte le comunicazioni e le notifiche di posta. Coordina e mantiene in sé lo stato persistente delle connessioni, la logica di autenticazione verso il provider SMTP esterno, il pooling delle connessioni e le code di invio dei messaggi, evitando il sovraccarico derivante dall'apertura di connessioni multiple e ridondanti verso i server di posta.

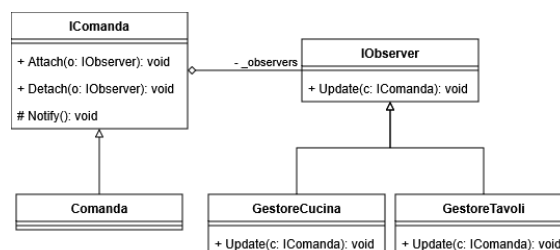


5.2 Pattern Observer con esempi

Il pattern Observer è un pattern comportamentale utilizzato per risolvere quelle situazioni in cui il cambiamento di stato di un oggetto (il Soggetto) deve essere notificato automaticamente a uno o più oggetti dipendenti (gli Osservatori), senza che il Soggetto sia conscio dell'identità specifica di tali osservatori o delle modalità con cui essi debbano elaborare l'aggiornamento. Questo disaccoppiamento evita di stabilire un legame forte tra i moduli, il quale renderebbe i componenti rigidi e difficilmente riutilizzabili.

Il pattern definisce una relazione uno-a-molti tra il Soggetto e gli oggetti da lui dipendenti. L'architettura si concretizza mediante l'uso di interfacce: il contratto del Soggetto mantiene un elenco astratto di osservatori e fornisce i metodi per l'agganciamento e lo sganciamento dinamico; gli osservatori, a loro volta, implementano un'interfaccia comune che espone un metodo di notifica che accetta come parametro il riferimento al soggetto stesso, consentendo la modalità *pull* per l'estrazione dei dati.

Un esempio pratico di questo pattern è rappresentato dalla gestione degli ordini all'interno di un ristorante, in cui una classe `Comanda` implementa l'interfaccia `IComanda`. Ogni volta che una nuova comanda viene emessa o modificata, essa invoca internamente il proprio metodo di notifica protetto, aggiornando automaticamente tutte le classi che implementano `IObserver`, ovvero il reparto `GestoreCucina` (per avviare la preparazione dei piatti) e il `GestoreTavoli` (per aggiornare lo stato di occupazione e il conto), senza che la comanda conosca i dettagli implementativi dei sistemi di destinazione.



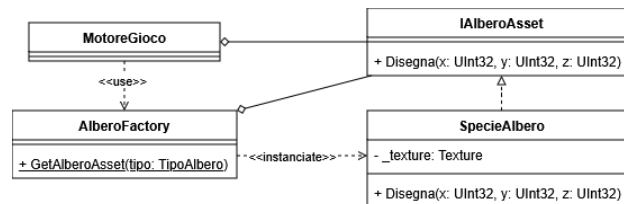
5.3 Pattern Flyweight con esempi

Il pattern Flyweight è un pattern strutturale progettato per ottimizzare l'uso della memoria Heap e ridurre il sovraccarico di allocazione quando il sistema deve gestire un numero elevatissimo di oggetti a granularità fine. Il principio cardine del pattern risiede nella condivisione di istanze preesistenti anziché nella creazione continua di nuovi oggetti tramite l'operatore `new`. Un oggetto Flyweight condiviso può essere utilizzato simultaneamente da più elementi del programma, rimanendo del tutto indistinguibile rispetto a un oggetto allocato singolarmente.

Gli oggetti Flyweight vengono gestiti centralmente da una fabbrica (*Flyweight Factory*) che si occupa di indicizzarli all'interno di una struttura dati di cache e di instanziarli solo se non sono già presenti nel sistema. Per rendere possibile la condivisione in contesti applicativi differenti, il pattern impone la scomposizione dello stato dell'oggetto in due componenti logiche nette:

- **Stato Intrinseco:** Comprende tutte le informazioni pesanti, costanti e indipendenti dal contesto. Questo stato è strutturalmente immutabile, appartiene all'istanza condivisa e viene memorizzato stabilmente all'interno del Flyweight nell'Heap.
- **Stato Estrinseco:** Comprende le informazioni variabili che dipendono strettamente dal contesto specifico di utilizzo. Questo stato non può essere condiviso, viene memorizzato e calcolato dal client esternamente e deve essere passato al Flyweight come parametro dei suoi metodi solo al momento dell'esecuzione del comportamento richiesto.

Un esempio ideale di questo pattern si riscontra nella gestione degli asset grafici all'interno di un motore di gioco tridimensionale, come il rendering di una foresta composta da migliaia di alberi. Applicando il pattern Flyweight, il motore di gioco istanzia un unico modello di albero per ogni specie botanica presente (ad esempio, un solo oggetto per l'abete, uno per la quercia). Lo stato intrinseco, memorizzato una sola volta nel Flyweight, conterrà la mesh geometrica e la texture della corteccia e delle foglie. Lo stato estrinseco, invece, consisterà unicamente nelle coordinate spaziali tridimensionali (posizione X, Y, Z) di ogni singolo albero nella mappa del mondo. Il client (il ciclo di rendering del gioco) scorrerà l'elenco delle posizioni logiche degli elementi e, per ciascuno di essi, invocherà il metodo di disegno del Flyweight corrispondente alla specie d'albero, passandogli le coordinate di calcolo come parametri dinamici.



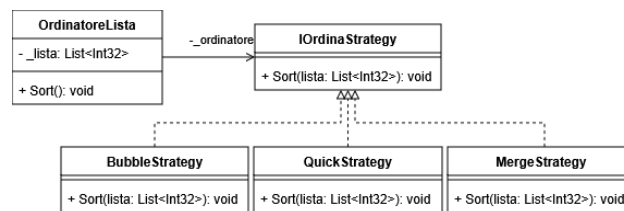
5.4 Pattern Strategy con esempi

Il pattern Strategy è un pattern comportamentale progettato per definire una famiglia di algoritmi, incapsulare ciascuno di essi all'interno di una classe specifica e renderli reciprocamente intercambiabili. Questa impostazione consente di variare la logica algoritmica in modo dinamico a runtime in base alle necessità del contesto, senza che l'entità chiamante debba essere modificata o sia a conoscenza dei dettagli implementativi interni. L'adozione del pattern favorisce il rispetto del principio *Open/Closed*, poiché l'introduzione di un nuovo comportamento non richiede la riscrittura del codice esistente, ma semplicemente l'estensione della gerarchia mediante una nuova classe concreta.

L'architettura del pattern si articola su tre componenti logiche fondamentali:

- **Interfaccia Strategy:** Definisce il contratto sintattico comune a tutti gli algoritmi della famiglia, esponendo la firma del metodo di esecuzione.
- **Strategie Concrete:** Classi distinte che realizzano l'interfaccia comune, ognuna delle quali implementa una specifica variante dell'algoritmo.
- **Contesto:** La classe che usufruisce del comportamento dinamico. Mantiene al suo interno un riferimento astratto verso l'interfaccia della strategia e delega a essa l'esecuzione dell'algoritmo, offrendo un metodo pubblico per consentire la sostituzione della strategia a runtime.

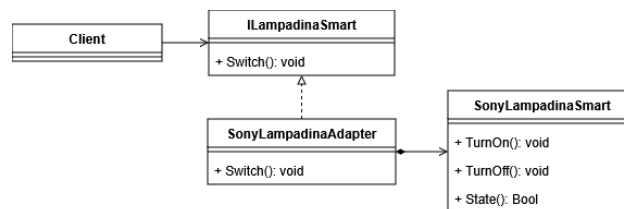
Un esempio classico e intuitivo di questo pattern si riscontra nella gestione degli algoritmi di ordinamento applicati a una collezione di dati. Si supponga di disporre di una classe manager denominata `OrdinatoreLista` (il Contesto) che ha il compito di organizzare sequenzialmente gli elementi di un elenco. A seconda delle dimensioni della lista o dei vincoli di sistema, potrebbe essere preferibile adottare strategie di ordinamento differenti: un algoritmo *Bubble Sort* per collezioni di piccole dimensioni, un *Quick Sort* per massimizzare la velocità su grandi volumi di dati, oppure un *Merge Sort* per garantire la stabilità dell'ordinamento.



5.5 Pattern Adapter con esempi

Il pattern Adapter (noto anche come *Wrapper*) è un pattern strutturale che permette di convertire l'interfaccia originale di una classe in un'altra interfaccia attesa dal client. Questo meccanismo consente la collaborazione e l'interoperabilità tra classi che, altrimenti, non potrebbero lavorare insieme a causa di incompatibilità tra i rispettivi contratti sintattici o firme dei metodi. L'adozione del pattern è particolarmente utile quando si desidera riutilizzare componenti preesistenti, moduli di terze parti o librerie proprietarie di produttori hardware senza poterne alterare il codice sorgente originale.

Un esempio pratico di questo pattern si riscontra nei sistemi di domotica, in cui un'applicazione di controllo centralizzata deve poter gestire dispositivi intelligenti di produttori diversi che rispondono a standard e protocolli hardware eterogenei.

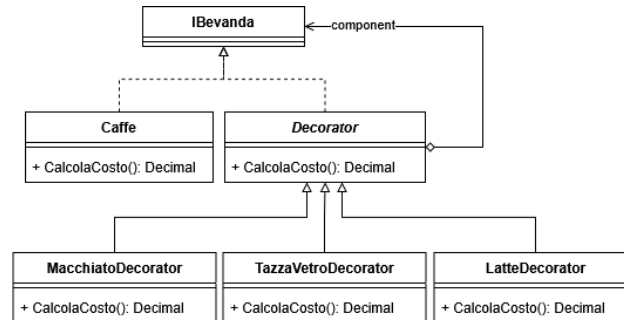


5.6 Pattern Decorator con esempi

Il pattern Decorator è un pattern strutturale che permette di aggiungere responsabilità a un oggetto in modo dinamico e trasparente, offrendo un'alternativa flessibile all'ereditarietà. L'architettura prevede un'interfaccia base per il componente, una classe astratta `Decorator` che la implementa e mantiene un riferimento privato all'oggetto da modificare, e una serie di sottoclassi concrete per ogni specifico modificatore.

Un esempio immediato è il calcolo del prezzo di un **Caffè**, che implementa l'interfaccia **IBevanda** e il metodo pubblico **calcolaCosto()**.

Per comporre liberamente le varianti a runtime (come un caffè macchiato, in tazza di vetro o con latte) senza creare un'esplosione di sottoclassi rigide, si introduce la classe astratta **OpzioneDecorator**, la quale realizza **IBevanda**. Le classi concrete **MacchiatoDecorator**, **TazzaVetroDecorator** e **LatteDecorator** ereditano dal decoratore base: quando viene invocato **calcolaCosto**, delegano il calcolo iniziale all'oggetto interno **_bevanda** e sommano dinamicamente il proprio sovrapprezzo.

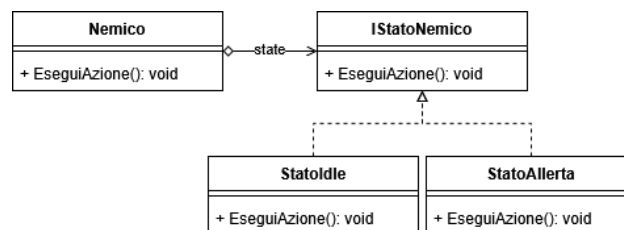


5.7 Pattern State con esempi

Il pattern State è un pattern comportamentale che permette a un oggetto di variare il proprio comportamento quando cambia il suo stato interno, dando l'impressione che l'oggetto abbia cambiato classe. Invece di mantenere enormi costrutti condizionali (come **switch** o **if-else**) legati a una variabile di stato, il pattern localizza i comportamenti specifici all'interno di classi indipendenti. Viene definita un'interfaccia comune per tutti gli stati associati a un contesto; quest'ultimo mantiene un riferimento all'istanza dello stato corrente e le delega l'esecuzione delle operazioni.

Un esempio ideale si riscontra nella gestione dell'intelligenza di un nemico all'interno di un videogioco. Si supponga di avere la classe di contesto **Nemico** che deve modificare le proprie azioni in base alla situazione corrente. Gli stati possibili sono modellati tramite le classi concrete **IdleState**, **AllertaState** e **CombattimentoState**, le quali realizzano l'interfaccia comune **IStatoNemico**.

L'interfaccia definisce i metodi di interazione. Quando l'entità **Nemico** riceve un input dal ciclo di gioco, non esegue verifiche condizionali interne, ma invoca semplicemente il metodo sull'oggetto **IStatoNemico** correntemente referenziato.



5.8 Pattern Composite con esempi

Il pattern Composite è un pattern strutturale che permette di comporre oggetti in strutture ad albero per rappresentare gerarchie parte-tutto, consentendo al client di trattare in modo uniforme oggetti singoli e contenitori.

L'architettura si articola su tre componenti principali:

- **Component:** Classe astratta o interfaccia comune che dichiara le operazioni per tutti gli elementi.
- **Leaf (Foglia):** Classe concreta che realizza gli oggetti terminali privi di figli.
- **Composite (Composto):** Classe concreta che definisce i nodi contenitori, memorizzando i figli in una collezione privata.

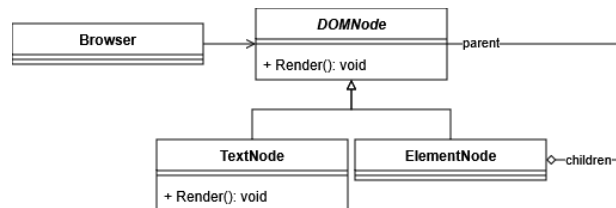
Ogni componente mantiene un riferimento al proprio elemento padre, e tutti i nodi con il medesimo parent costituiscono gli unici figli di quel determinato componente.

Nella progettazione del **Component** si delineano tre trade-off fondamentali:

- **Massimizzazione dell'interfaccia:** Consiste nell'inserire nel **Component** tutte le operazioni necessarie ai client, in modo che questi vedano esclusivamente l'astrazione di alto livello.
- **Trasparenza:** Deriva dalla massimizzazione. Dichiarando i metodi di gestione dei figli direttamente nel **Component**, il client tratta ogni oggetto in modo uniforme. Il rischio è la perdita di sicurezza, poiché il client potrebbe tentare azioni prive di senso sulle foglie (es. aggiungere un tag figlio a un nodo di testo puro).
- **Sicurezza:** Approccio opposto. Le operazioni sui figli vengono rimosse dal **Component** e inserite esclusivamente nel **Composite**. Si ottiene un sistema sicuro a compile-time, ma il client perde l'uniformità e deve gestire esplicitamente due interfacce differenti.

Un esempio reale e universale è il Document Object Model (DOM) HTML. La classe astratta base **DOMNode** espone il metodo pubblico **Render()**.

La classe concreta **TextNode** agisce da *Leaf* e implementa **Render()** convertendo in grafica la stringa di testo testuale. La classe concreta **ElementNode** funge da *Composite*, incapsulando una collezione privata ed espone i metodi di gestione della struttura. Quando il client o il browser invoca **Render()** su un **ElementNode**, questo esegue la propria logica di formattazione e cicla ricorsivamente sulla collezione interna, invocando lo stesso metodo su ogni sotto-nodo o testo contenuto.



5.9 Pattern Visitor con esempi

Il pattern Visitor è un pattern comportamentale che permette di separare un algoritmo dalla struttura di oggetti su cui opera, consentendo di definire nuove operazioni senza dover modificare le classi degli elementi esistenti. Tutto il codice relativo a una singola operazione viene incapsulato all'interno di una classe concreta **Visitor**, la quale esegue la propria logica visitando gli elementi della struttura.

L'architettura del pattern si basa sul meccanismo del *Double Dispatch*, ovvero un meccanismo di accettazione prima e visita dopo, e comprende:

- **Visitor:** Interfaccia o classe astratta che dichiara un metodo di visita sovraccaricato per ciascuna classe concreta di elementi presente nella gerarchia.
- **ConcreteVisitor:** Classe concreta che implementa una specifica operazione o calcolo da eseguire sull'intera struttura di oggetti.

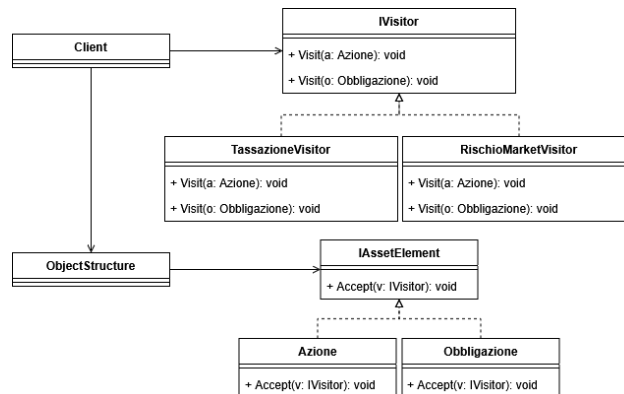
- **Element:** Interfaccia o classe astratta che definisce l'operazione di accettazione mediante il metodo pubblico `Accept(visitor: IVisitor)`.
- **ConcreteElement:** Classe concreta che implementa il metodo di accettazione, limitandosi a ridirigere la chiamata verso il visitor tramite la sintassi `visitor.Visit(this)`.

La struttura che ospita gli elementi può essere un'architettura ricorsiva basata sul pattern **Composite** o una generica collezione di oggetti enumerabili.

Un esempio pratico si riscontra nei sistemi di reportistica e analisi finanziaria per portafogli di investimento. Si supponga di disporre di una collezione di asset patrimoniali che implementano l'interfaccia **IAssetElement**. Gli elementi concreti sono rappresentati dalle classi **Azione** e **Obbligazione**.

Se si desidera introdurre due nuove operazioni distinte, come il calcolo delle tasse correnti e la valutazione del rischio di mercato, non si modifica il codice interno delle classi degli asset. Si definisce invece l'interfaccia **IAssetVisitor** che espone due varianti del metodo di visita.

Successivamente si creano le classi concrete **TassazioneVisitor** e **RischioMarketVisitor**. Quando l'applicazione client cicla sulla collezione di asset, invoca il metodo `Accept` su ciascun oggetto passando l'istanza del visitor scelto. L'elemento concreto risponde invocando a sua volta il metodo `Visit` sul visitor e passando se stesso (`this`) come argomento, consentendo al visitor di accedere ai membri pubblici protetti dall'incapsulamento ed eseguire l'algoritmo specifico.



5.10 Pattern Abstract Factory con esempi

Il pattern Abstract Factory è un pattern creazionale che permette di creare famiglie di oggetti correlati o dipendenti tra loro, senza che il client debba specificare le loro classi concrete. Sviluppa un sistema indipendente per la generazione di prodotti tra diverse varianti o famiglie, garantendo il vincolo che gli oggetti appartenenti alla stessa famiglia siano sempre utilizzati insieme. Il pattern promuove l'isolamento delle classi concrete, facilita la sostituzione di un'intera famiglia di prodotti a runtime e garantisce la coerenza tra i componenti creati. Tale pattern rende però difficile l'aggiunta di nuove tipologie di prodotti, poiché richiederebbe la modifica di tutte le factory esistenti.

L'architettura del pattern si articola su quattro elementi principali:

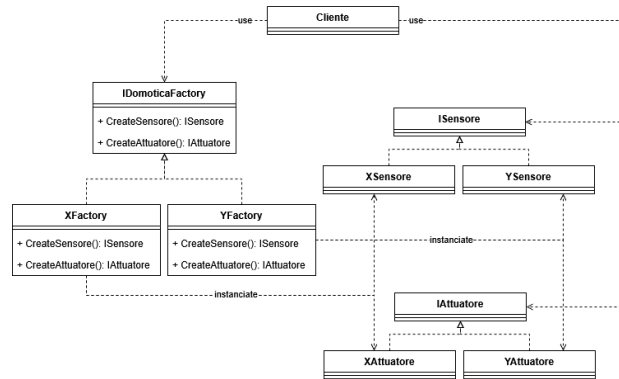
- **AbstractFactory:** Interfaccia che dichiara un insieme di metodi per la creazione di ciascuno dei prodotti astratti.
- **ConcreteFactory:** Classi concrete che implementano le operazioni di creazione per generare i prodotti specifici di una determinata famiglia.
- **AbstractProduct:** Interfacce o classi astratte per una serie di prodotti distinti ma correlati che compongono la famiglia.

- **ConcreteProduct:** Implementazioni concrete dei prodotti, create esclusivamente dalla corrispondente fabbrica concreta.

Un esempio pratico è la configurazione di ecosistemi di dispositivi per la domotica intelligente, divisi per protocollo di comunicazione.

I componenti indipendenti sono definiti dalle interfacce astratte dei prodotti, come **ISensore** e **IAttuatore**. Le implementazioni concrete per una famiglia saranno **XSensore** e **XAttuatore**, mentre per l'altra famiglia saranno **YSensore** e **YAttuatore**.

Per astrarre la creazione e garantire che un modulo di controllo non mescoli erroneamente dispositivi incompatibili, si definisce l'interfaccia **IDomoticaFactory** che dichiara i metodi di fabbricazione. Le classi concrete **XFactory** e **YFactory** realizzano l'interfaccia della factory. Il client interagisce esclusivamente tramite le interfacce astratte.



5.11 Pattern MVP ed MVC

Il pattern MVC (Model-View-Controller) è un pattern architetturale finalizzato alla separazione delle responsabilità all'interno di un'applicazione con interfaccia grafica, strutturato su tre elementi logici distinti:

- **Modello (Model):** Rappresenta il nucleo applicativo e gestisce i dati logicamente correlati insieme alle relative regole di business. Esso risponde alle interrogazioni sullo stato attuale e alle istruzioni di modifica dei dati provenienti dal Controller. Inoltre, il Modello genera eventi di notifica quando il suo stato interno cambia, registrando gli oggetti interessati secondo una logica tipica del pattern Observer.
- **Vista (View):** Gestisce l'area visualizzabile sullo schermo e si occupa della rappresentazione grafica dei dati forniti dal Modello. La Vista si registra direttamente sul Modello per ricevere le notifiche dei suoi cambiamenti di stato; al ricezione di un evento, essa interroga in modalità *pull* il Modello per aggiornare l'interfaccia utente.
- **Controller:** Gestisce gli input e le interazioni dell'utente (come click o pressione di tasti) provenienti dalla Vista, li interpreta e invia i relativi comandi di modifica al Modello o comandi di visualizzazione alla Vista stessa.

Il pattern MVP (Model-View-Presenter), invece, nasce come evoluzione per incrementare il disaccoppiamento e facilitare i test dell'interfaccia grafica. Questo modello rimuove completamente la comunicazione diretta e la dipendenza bidirezionale tra la Vista e il Modello.

Nello schema MVP, la Vista diventa totalmente passiva e delega tutta la logica di presentazione al Presenter. La notifica delle variazioni del Modello e la conseguente modifica dei dati visualizzati avvengono esclusivamente attraverso la mediazione del Presenter. La Vista interagisce unicamente con il Presenter, il quale aggiorna lo stato del Modello, ne intercetta i cambiamenti e imposta i dati sulla Vista sincronizzandola in modo esplicito.

6 Version Control System

6.1 Modello LMU nei VCS con vantaggi e svantaggi

Il modello *Lock-Modify-Unlock* (LMU) è una strategia tradizionale di gestione della concorrenza nei sistemi di controllo versione (VCS) di tipo centralizzato. In questo modello, l'accesso in scrittura ai file del repository è rigorosamente mutualmente esclusivo: a un solo utente alla volta è concesso il permesso di modificare una determinata risorsa. Il flusso di lavoro prevede che ogni sviluppatore, per operare su un file, debba acquisirne il blocco (*Lock*), effettuare le modifiche locali (*Modify*) e infine rilasciare la risorsa (*Unlock*) per rimetterla a disposizione del team.

Sebbene l'approccio garantisca una gestione deterministica dei file non unificabili testualmente, esso introduce una serie di svantaggi critici:

- **Blocchi amministrativi e dimenticanze:** Se un utente acquisisce il lock su un file e si allontana o dimentica di rilasciarlo, la risorsa rimane inaccessibile a tempo indeterminato per l'intero team, paralizzando lo sviluppo.
- **Inibizione della concorrenza parallela:** Il modello impedisce modifiche simultanee anche quando due programmatori devono operare su regioni di codice completamente diverse, indipendenti e non connesse dello stesso file.
- **Rischio di stallo logico (Deadlock):** Si verifica una situazione di stallo quando l'utente *X* acquisisce il lock sul file *A* e necessita del file *B* per completare il lavoro, mentre l'utente *Y* detiene il lock sul file *B* e attende il rilascio del file *A*.
- **Impossibilità di lavoro offline:** Poiché l'acquisizione e il rilascio dei lock richiedono un'interazione diretta e atomica con il server centrale, gli sviluppatori non possono avanzare nel proprio lavoro in assenza di una connessione di rete stabile.

Di contro, l'unico vero vantaggio del modello LMU risiede nella gestione dei file binari non testuali (come immagini, modelli 3D o file audio compresi negli asset di progetto). Poiché tali file non supportano le operazioni di fusione automatica (*Merge*), il lock preventivo costituisce l'unico meccanismo sicuro per evitare conflitti distruttivi e sovrascritture accidentali.

6.2 Modello CMM nei VCS con vantaggi e svantaggi

Il modello *Copy-Modify-Merge* (CMM) è la strategia di gestione della concorrenza nativa dei sistemi di controllo versione (VCS) moderni e distribuiti. Esso non utilizza alcun tipo di blocco esclusivo sui file. Il flusso di lavoro permette a più sviluppatori di copiare in locale l'intero repository, modificare in parallelo gli stessi file e, in fase di consegna (*Check-in*), fondere le proprie modifiche con quelle degli altri membri del team tramite un'operazione di fusione (*Merge*).

Questo meccanismo si basa sulla marcatura temporale o sequenziale dei file. Se la versione del file sul server è più avanzata rispetto alla copia di partenza usata dallo sviluppatore per le sue modifiche locali, il sistema impedisce il completamento del *Check-in* finché l'utente non sincronizza il proprio ambiente ed esegue il merge. Gli esiti possibili di un'operazione di merge testuale sono due:

- **Successo (Automatico):** Si verifica quando le modifiche concorrenti interessano linee di codice o regioni del file totalmente distinte. Il VCS unisce i contributi automaticamente senza alterare la struttura sintattica del file.
- **Conflitto:** Si verifica quando due o più utenti modificano le medesime righe di testo. In questo scenario, il sistema sospende l'operazione e richiede l'intervento manuale dello sviluppatore per risolvere l'ambiguità.

Oltre ai conflitti puramente testuali, l'approccio CMM introduce il rischio di **conflitti semantici**. Un conflitto semantico si verifica quando il VCS esegue con successo un merge automatico dal punto di vista sintattico (poiché le modifiche insistono su righe di codice o file differenti), ma l'unione dei due contributi altera o corrompe la logica di funzionamento complessiva del programma. Un esempio classico si ha quando uno sviluppatore modifica la firma di un metodo o ne altera gli effetti collaterali in un modulo, mentre un secondo sviluppatore ne inserisce un'invocazione (formalmente corretta ma logicamente errata) in un altro modulo del progetto. Poiché il sistema di controllo versione ragiona esclusivamente a livello di stringhe di testo e non comprende la semantica del linguaggio, non è in grado di rilevare queste anomalie, le quali si manifesteranno soltanto in fase di compilazione o, nello scenario peggiore, come bug a runtime durante l'esecuzione dei test.

L'adozione del modello CMM offre significativi vantaggi operativi, ma introduce anche alcuni compromessi:

- **Vantaggi:** Massimizza la produttività del team consentendo lo sviluppo parallelo e simultaneo sullo stesso codice; azzerà i colli di bottiglia causati da blocchi dimenticati o persistenti; permette il lavoro in modalità completamente offline sulla propria copia locale.
- **Svantaggi:** Sposta il carico della risoluzione dei conflitti (a volte complessi e legati alla logica dell'applicazione) direttamente sugli sviluppatori in fase di merge; non offre alcuna protezione nativa contro i conflitti di tipo semantico; risulta inefficace o distruttivo se applicato a file binari non testuali (come immagini, archivi o asset multimediali), per i quali la fusione automatica del testo è strutturalmente impossibile.

Secondo modulo

7 Modelli, Linguaggi e Modelli di Processo di sviluppo

7.1 Il modello a cascata e le sue criticità

Il modello a cascata è un ciclo di vita del software basato sul presupposto economico che introdurre cambiamenti o correggere errori nelle fasi avanzate dello sviluppo comporti costi proibitivi. Di conseguenza, il modello impone che ogni fase sia eseguita in maniera esaustiva e si concluda definitivamente prima di passare alla successiva, senza prevedere flussi di ritorno. I risultati in uscita da ciascuno stadio prendono il nome di *semilavorati* e devono essere rigorosamente documentati e approvati entro scadenze temporali prestabilite. Le fasi sequenziali che caratterizzano questo ciclo di vita sono:

- **Studio di fattibilità:** Valutazione preventiva dei costi, dei benefici e dei vincoli tecnologici o di tempo del progetto.
- **Analisi dei requisiti:** Identificazione, negoziazione e formalizzazione di tutte le funzionalità e i vincoli del sistema.
- **Progettazione:** Definizione dell'architettura logica, strutturale e dei dati del software.
- **Implementazione:** Traduzione del progetto in codice sorgente e sviluppo dei singoli moduli.
- **Collaudo:** Verifica e validazione del sistema integrato per garantirne la conformità ai requisiti.
- **Manutenzione:** Gestione dei bug residui, aggiornamenti e adattamenti del software rilasciato nel tempo.

L'approccio a cascata poggia su due assiomi idealizzati che, nei contesti industriali reali, si rivelano essere le sue principali criticità:

- **Comprensione totale dei requisiti:** Presuppone che il cliente sia in grado di esplicitare e descrivere dettagliatamente tutte le proprie esigenze all'inizio del progetto, rendendo i requisiti stabili e immutabili.
- **Prevedibilità del design:** Assume che sia possibile progettare in modo perfetto e onnicomprensivo l'intero sistema software prima ancora di scriverne una singola riga di codice.

Per mitigare l'inflessibilità del modello originario, sono state storicamente introdotte delle varianti evolute. La prima prevede l'inserimento di cicli di *retroazione limitati a un solo livello*, consentendo di tornare esclusivamente alla fase immediatamente precedente in caso di anomalie circoscritte. La seconda variante prevede l'integrazione di un prototipo usa e getta (*Throw-away Prototype*), sviluppato rapidamente per fornire al committente una base concreta su cui validare e raffinare le specifiche prima di avviare la cascata vera e propria. Tuttavia, lo sviluppo e lo scarto di un prototipo introducono un sovraccarico di tempi e costi che, di fatto, annulla l'efficienza economica teorica che il modello a cascata standard si prefiggeva di garantire.

7.2 Il modello a cascata e il modello iterativo

Il modello a cascata è un ciclo di vita del software sequenziale e lineare, basato sul presupposto economico che introdurre cambiamenti o correggere errori nelle fasi avanzate dello sviluppo comporti costi proibitivi. Di conseguenza, il modello impone che ogni fase (dallo studio di fattibilità fino al collaudo) sia eseguita in maniera esaustiva e si concluda definitivamente prima di passare alla successiva, senza prevedere flussi di ritorno. I risultati in uscita da ciascuno stadio prendono il nome di *semilavorati* e devono essere rigorosamente documentati e approvati entro scadenze temporali prestabilite (*milestone*).

Questo processo dà per scontati due assiomi idealizzati che, nei contesti industriali reali, si rivelano essere le sue principali criticità:

- **Comprensione totale dei requisiti:** Presuppone che il cliente sia in grado di esplicitare e descrivere dettagliatamente tutte le proprie esigenze all'inizio del progetto, rendendo i requisiti stabili e immutabili.
- **Prevedibilità del design:** Assume che sia possibile progettare in modo perfetto e onnicomprensivo l'intero sistema software prima ancora di scriverne una singola riga di codice.

Per tentare di arginare tali problematiche, sono state storicamente introdotte delle varianti evolutive del modello a cascata. La prima prevede l'inserimento di cicli di *retroazione limitati a un solo livello*, consentendo di ritornare esclusivamente alla fase subito precedente in caso di anomalie circoscritte. La seconda variante prevede invece l'integrazione di un prototipo usa e getta (*Throw-away Prototype*), sviluppato rapidamente all'inizio del ciclo di vita per fornire al committente una base concreta su cui validare, raffinare e comprendere meglio le specifiche prima di avviare la cascata vera e propria. Tuttavia, lo sviluppo e il successivo scarto di questo prototipo introducono un sovraccarico di tempi e costi che finisce per annullare il risparmio economico complessivo che il modello a cascata teorico vorrebbe garantire.

Il modello iterativo nasce proprio per superare la rigidità di queste soluzioni intermedie, accettando l'imprevedibilità e la continua evoluzione dei requisiti come fattori intrinseci dello sviluppo del software. Invece di tentare di sviluppare l'intero sistema in un unico blocco lineare, questo approccio prevede un processo continuo di pianificazione, analisi, progettazione, codifica e test che si ripete ciclicamente.

Il sistema viene così costruito e raffinato pezzo per pezzo: ogni macro-ciclo operativo (*iterazione*) rilascia una versione funzionante ed eseguibile del software (un incremento), che si innesta sulla base di quella precedente. Questo permette di ottenere feedback costanti dal cliente e di convalidare le scelte architetturali.

7.3 Il modello di processo RUP

Il *Rational Unified Process* (RUP) è un modello di processo ibrido, iterativo e incrementale. Più che un singolo processo rigido, il RUP si definisce come un *framework* flessibile, capace di dare vita a istanze di processo differenti e personalizzate, configurandosi come una soluzione ideale per progetti software di grandi dimensioni. Il modello organizza lo sviluppo strutturando il ciclo di vita secondo tre prospettive o visioni complementari: dinamica, statica e pratica.

- **Dinamica:** Questa prospettiva esprime l'evoluzione del progetto nel tempo ed è suddivisa in quattro fasi sequenziali, ciascuna focalizzata sul raggiungimento di una specifica milestone fondamentale:
 - *Inception (Avvio):* Delinea nel modo più accurato possibile il *business case* e l'ambito del progetto. Comprende l'analisi del tipo di mercato e identifica le entità esterne (attori) che interagiranno con il sistema.
 - *Elaborazione:* Definisce la struttura complessiva del sistema attraverso l'analisi del dominio e una prima fase di progettazione dell'architettura. Per concludersi, deve soddisfare una serie di requisiti stringenti:
 - * Modello dei casi d'uso completo all'80%.
 - * Descrizione formale dell'architettura del sistema.
 - * Sviluppo di un'architettura eseguibile.
 - * Revisione e conferma del business case.
 - * Pianificazione del progetto complessivo.
 - *Construction (Costruzione):* Fase intensiva incentrata su progettazione di dettaglio, programmazione e test del programma. I moduli e le parti della struttura vengono sviluppati in parallelo e integrati progressivamente.
 - *Transition (Transizione):* Il sistema passa definitivamente dall'ambiente di sviluppo controllato a quello del client finale (deployment operativo).
- **Statica:** Questa prospettiva descrive le attività svolte durante il ciclo di vita, organizzate in sei flussi di lavoro (*workflow*) principali e tre di supporto, idealmente attivi in contemporanea ma con intensità variabili a seconda della fase temporale:
 - *Workflow principali:* Modellazione delle attività aziendali, Requisiti, Analisi e progetto, Implementazione, Test, Rilascio.
 - *Workflow di supporto:* Gestione della configurazione e delle modifiche, Gestione del progetto, Ambiente.
- **Pratica:** Riassume le linee guida ingegneristiche e le buone prassi raccomandate dal framework per mitigare i rischi e garantire il successo del prodotto:
 - Sviluppare il software ciclicamente (approccio iterativo).
 - Gestire e tracciare i requisiti lungo tutto il ciclo di vita.
 - Utilizzare architetture basate su componenti riusabili e disaccoppiati.
 - Creare modelli visivi del software (tramite diagrammi standard come UML).
 - Verificare costantemente la qualità del software.
 - Controllare e ispezionare le modifiche apportate al software.

8 Analisi dei requisiti

8.1 Tipologie di requisiti

Nell'ingegneria del software, i requisiti stabiliscono i servizi che il sistema deve fornire e i vincoli operativi a cui deve sottostare. È possibile definire e classificare i requisiti secondo due macro-livelli di astrazione:

- **Requisiti utente:** Vengono definiti in fase di colloquio con il committente e gli utenti finali. Sono espressi in linguaggio naturale coadiuvato da diagrammi ad alto livello (come i diagrammi dei casi d'uso), in maniera astratta. Il loro scopo principale è indicare quali servizi il sistema deve fornire e i vincoli generali sotto cui deve operare, senza scendere in dettagli tecnici, affinché siano comprensibili anche a personale non tecnico.
- **Requisiti di sistema:** Rappresentano una specifica tecnica approfondita che definisce nel dettaglio le funzioni, i servizi e i vincoli operativi del software. Fungono da base per il contratto stipulato tra cliente e sviluppatori e devono essere inseriti formalmente nel documento di specifica dei requisiti software. Questi si dividono a loro volta in tre categorie principali:
 - *Requisiti funzionali:* Descrivono esplicitamente quello che il sistema deve fare. Consistono in elenchi di servizi e funzionalità che specificano come il software debba reagire a determinati input, come debba comportarsi in situazioni specifiche o come esplicitamente *non* debba comportarsi. Per essere considerati validi, devono essere *completi* (includere tutte le funzionalità richieste) e *coerenti* (privi di scostamenti o contraddizioni interne).
 - *Requisiti non funzionali:* Definiscono le proprietà e i vincoli globali del sistema (come prestazioni, sicurezza, affidabilità e portabilità). Spesso sono difficilmente verificabili in modo quantitativo e possono trovarsi in contrapposizione con i requisiti funzionali o tra di loro (es. massimizzare la sicurezza riduce le prestazioni). Vanno studiati e analizzati attentamente e si suddividono in:
 - * **Requisiti del prodotto:** Specificano o limitano il comportamento e le proprietà complessive del software a runtime (es. velocità di esecuzione, usabilità, memoria massima utilizzata).
 - * **Requisiti organizzativi:** Derivano dalle politiche e dalle procedure interne dell'organizzazione del cliente o degli sviluppatori. Possono vincolare anche il processo di sviluppo adottato (es. l'obbligo di usare standard ISO, specifici linguaggi di programmazione o metodologie agili).
 - * **Requisiti esterni:** Conseguenza di fattori non provenienti direttamente dal sistema o dall'organizzazione aziendale, ma legati a vincoli di terze parti, legislativi o di mercato (es. normative sulla protezione dei dati personali come il GDPR, requisiti di interoperabilità con sistemi esterni o vincoli etici).
 - *Requisiti di dominio:* Derivano direttamente dal dominio di applicazione specifico del sistema e riflettono le caratteristiche e i vincoli di quel particolare settore. Essendo molto specialistici, introducono una terminologia propria e si riferiscono ai concetti fondamentali del dominio, risultando spesso impliciti per gli esperti del settore ma cruciali per gli sviluppatori.

9 Analisi dei requisiti di sicurezza

9.1 Principali categorie di requisiti per la sicurezza

I requisiti per la sicurezza sono spesso formulati in forma negativa (ad esempio, "il sistema non deve permettere...") e generalmente si dividono in:

- **Requisiti di identificazione:** definiscono come il sistema riconosce l'asserzione di identità da parte degli utenti o di altri sistemi (es. tramite username o ID).
- **Requisiti di autenticazione:** definiscono i meccanismi per verificare e convalidare l'identità dichiarata (es. tramite password, token, dati biometrici).
- **Requisiti di autorizzazione:** definiscono i privilegi, i ruoli e i permessi di accesso alle risorse del sistema per ciascun utente o processo autenticato.
- **Requisiti di immunità:** definiscono le modalità con cui il sistema deve proteggersi e resistere ad attacchi da parte di malware, virus, worm e altre minacce informatiche.
- **Requisiti di integrità:** garantiscono che i dati e le funzionalità del sistema non vengano modificati, cancellati o corrotti da entità non autorizzate o a causa di errori.
- **Requisiti di scoperta delle intrusioni:** definiscono le capacità del sistema di rilevare, registrare e segnalare tentativi di accesso non autorizzato o attività sospette in tempo reale.
- **Requisiti di non-ripudio:** garantiscono che una transazione o un'azione eseguita nel sistema non possa essere negata dall'autore (es. tramite firme digitali e log immutabili).
- **Requisiti di riservatezza:** assicurano che le informazioni siano accessibili esclusivamente ai soggetti esplicitamente autorizzati a prenderne visione.
- **Requisiti di controllo della protezione:** definiscono i meccanismi per monitorare, verificare e aggiornare lo stato e l'efficacia delle misure di sicurezza implementate.
- **Requisiti di protezione della manutenzione del sistema:** garantiscono che le attività di manutenzione, aggiornamento o riparazione (sia locali che remote) non introducano vulnerabilità o accessi non controllati.

9.2 Il ciclo di vita della valutazione del rischio

Il ciclo di vita della valutazione del rischio costituisce il nucleo operativo dell'intero processo di gestione del rischio. Partendo dalla profonda conoscenza dell'architettura del sistema, delle tecnologie adottate e dell'organizzazione dei dati, questo ciclo si articola in un processo iterativo volto a mappare e quantificare le minacce.

Il ciclo di vita si sviluppa tipicamente attraverso le seguenti fasi macroscopiche:

1. **Identificazione dei beni:** Individuazione e classificazione dei beni di valore del sistema (hardware, software, dati, processi critici) che hanno la maggiore probabilità o impatto in caso di compromissione.

2. **Identificazione delle vulnerabilità e delle minacce:** Analisi del sistema per far emergere le debolezze intrinseche (spesso ereditate involontariamente dalle prime fasi di progettazione) e messa in relazione di queste vulnerabilità con i potenziali vettori di attacco o minacce ambientali.
3. **Stima e ponderazione del rischio:** Calcolo del livello di rischio combinando la probabilità che una minaccia sfrutti una determinata vulnerabilità e la gravità del danno (impatto) che ne deriverebbe per l'organizzazione.

Il risultato finale di questo ciclo non è statico, ma si traduce in un insieme di decisioni strategiche e contro-misure. Tali decisioni influenzano direttamente la successiva progettazione o implementazione del sistema, guidano lo sviluppo di patch e possono arrivare a limitare o normare i modi stessi in cui il sistema viene utilizzato dagli utenti operativi.

10 Progettazione per la sicurezza

10.1 Linee guida di progettazione nella sicurezza

La progettazione di sistemi software sicuri richiede l'adozione di un approccio strutturato, poiché contesti operativi differenti impongono misure di protezione specifiche. Per orientare gli sviluppatori nell'architettura dei sistemi, sono state formalizzate delle linee guida generali ad ampia applicazione che mirano a ridurre la superficie di attacco e a garantire la resilienza del software:

- **Basare le decisioni di sicurezza su una politica esplicita:** Ogni scelta progettuale deve fare riferimento a un documento formale di *Security Policy*, il quale definisce chiaramente gli obiettivi e i requisiti di sicurezza del sistema senza vincolarsi a specifici meccanismi tecnologici. Queste direttive vengono incorporate nell'architettura logica attraverso un'autorità di sicurezza (*Security Authority*). Le principali politiche di riferimento includono:
 - *Identity policies*: Definiscono le modalità di autenticazione, identificazione e tracciamento degli utenti e dei servizi autorizzati a interagire con il sistema.
 - *Access control policies*: Stabiliscono le regole di autorizzazione, associando i singoli profili utente ai permessi di lettura, scrittura o esecuzione sulle risorse.
 - *Content-specific policies*: Regolano il trattamento, la cifratura e la classificazione dei dati sensibili in base alla loro natura o criticità.
 - *Network and infrastructure policies*: Determinano i protocolli di comunicazione sicuri e i vincoli di instradamento del traffico sulla rete.
 - *Regulatory policies*: Garantiscono la conformità del sistema rispetto a standard normativi, legali e di settore.
 - *Advisory and information policies*: Definiscono le procedure di audit, la gestione dei log e i protocolli di notifica in caso di rilevamento di incidenti o vulnerabilità.
- **Evitare il singolo punto di fallimento (Single Point of Failure - SPoF):** L'architettura non deve mai affidare l'intera integrità del sistema a un unico componente o a un solo meccanismo di controllo. È necessario implementare una strategia di *Defense in Depth* (difesa in profondità), strutturando controlli di sicurezza stratificati e indipendenti su più livelli.

- **Fallire in modo sicuro (Fail-Safe Defaults):** In caso di errore critico, eccezione imprevista o crash di un componente, il sistema deve transitare automaticamente verso uno stato di massima protezione. Non deve mai essere consentito a un attaccante di accedere a dati riservati o di bypassare i controlli a causa di un'interruzione anomala del codice.
- **Bilanciare sicurezza e usabilità:** L'introduzione di meccanismi di protezione tende inevitabilmente a ridurre l'usabilità del software e a complicare l'esperienza utente. Una progettazione efficace deve mediare tra il livello di rischio accettabile e la fluidità operativa, evitando restrizioni eccessive che potrebbero spingere gli utenti ad aggirare i controlli stessi.
- **Essere consapevoli dell'esistenza dell'ingegneria sociale:** La progettazione deve prevedere l'errore o la manipolazione degli operatori umani, implementando controlli software stringenti, restrizioni sui privilegi e procedure di verifica incrociata per limitare l'impatto di azioni umane non autorizzate.
- **Usare ridondanza e diversità per ridurre i rischi:** Le diverse istanze o repliche del sistema dovrebbero essere diversificate.
- **Validare tutti gli input:** Qualsiasi dato proveniente da una sorgente esterna deve essere rigorosamente controllato, sanitizzato e tipizzato prima di essere elaborato dal sistema.
- **Dividere in compartimenti i beni (Principle of Least Privilege):** Il sistema deve essere strutturato in moduli e compartimenti isolati. Gli utenti e i processi interni devono disporre esclusivamente dei privilegi minimi necessari a completare la propria attività specifica (*Need-to-Know basis*).
- **Progettare per il deployment:** L'architettura deve semplificare la configurazione sicura del software in fase di installazione e manutenzione in ambiente di produzione. Questa macro-linea guida si articola in quattro direttive operative fondamentali:
 - *Includere supporto per visionare e analizzare le configurazioni:* Il sistema deve esporre strumenti nativi che permettano agli amministratori di ispezionare, validare e verificare in ogni momento lo stato corrente di tutte le impostazioni di sicurezza attive, facilitando l'individuazione di disallineamenti o falle.
 - *Minimizzare i privilegi di default:* Il software deve essere distribuito applicando di base una configurazione improntata al principio del minimo privilegio (*Least Privilege*). Gli account predefiniti, i servizi di background e i moduli accessori devono disporre del livello minimo di autorizzazioni indispensabile per il funzionamento iniziale, riducendo l'impatto di un'eventuale compromissione immediata.
 - *Localizzare le impostazioni di configuration:* Tutte le opzioni relative alla sicurezza, ai permessi e ai parametri di rete devono essere centralizzate e strutturate all'interno di un unico punto logico o di file di configurazione ben definiti e isolati. Questo evita la frammentazione dei parametri all'interno del codice sorgente, riducendo drasticamente il rischio di errori umani in fase di deployment.
 - *Fornire modi per rimediare a vulnerabilità di sicurezza:* L'architettura deve prevedere meccanismi integrati e sicuri per l'applicazione tempestiva di patch, aggiornamenti e correzioni firmware o software. Devono essere documentate e predisposte procedure chiare per sanare tempestivamente eventuali falle scoperte dopo il rilascio, minimizzando i tempi di esposizione del sistema.
- **Progettare per il ripristino:** Assumendo che nessuna difesa sia infallibile, il sistema deve integrare fin dalle prime fasi di progettazione strategie di ripristino rapido. Ciò include meccanismi automatizzati di backup, isolamento tempestivo dei nodi compromessi, tracciamento sicuro degli eventi tramite log non modificabili e procedure per il rollback controllato a versioni stabili del software.

10.2 Test di sicurezza: Black-box e White-box

Al fine di valutare la sicurezza di un sistema e testarne la resilienza fino al punto di rottura, vengono impiegate due metodologie d'indagine complementari che differiscono per il livello di conoscenza preliminare dell'infrastruttura concesso all'analista:

- **Black-box Testing:** Questa tecnica assume la totale assenza di conoscenza pregressa riguardo all'architettura interna, alla logica applicativa o alle tecnologie del sistema sotto esame. Il tester affronta il target simulando fedelmente la prospettiva e l'operato di un attaccante esterno reale. Di conseguenza, l'indagine non si limita all'ispezione delle debolezze del codice esposto, ma si estende a test mirati a livello infrastrutturale e di rete, volti a identificare servizi attivi vulnerabili, difetti di configurazione dei server, porte aperte e falle nel perimetro difensivo.
- **White-box Testing:** Questa metodologia presuppone, al contrario, la completa e trasparente conoscenza del sistema da verificare. L'analista dispone di pieno accesso alla documentazione tecnica, ai file di configurazione degli ambienti e, soprattutto, all'intero codice sorgente del software. In questo scenario, le attività comprendono l'analisi statica e dinamica del codice e la scrittura di test d'integrazione e di unità specifici. Avvalendosi di strumenti di tracciamento e *tool di debug*, il white-box testing è particolarmente efficace nell'individuare vulnerabilità logiche interne profondamente radicate nel codice, come la mancanza di validazione o sanitizzazione degli input, le corse critiche (*race conditions*), i problemi di concorrenza e i difetti di gestione della memoria come i *memory leak*.

10.3 Capacità di sopravvivenza del sistema

La capacità di sopravvivenza (*Survivability*) di un sistema è definita come la sua attitudine a continuare a erogare i servizi essenziali e critici anche in presenza di attacchi informatici esterni, intrusioni o a seguito del danneggiamento intenzionale o accidentale di parti della sua infrastruttura. Garantire la sopravvivenza costituisce una priorità ingegneristica dell'intero ciclo di vita del sistema.

La disponibilità costante dei servizi critici rappresenta l'essenza stessa della sopravvivenza. Questo presuppone una conoscenza analitica approfondita di quali siano i servizi maggiormente esposti e compromettibili, quale sia la soglia qualitativa minima da mantenere in condizioni di emergenza, come proteggere tali asset e in quale modo strutturarne il ripristino tempestivo.

Il *Survivability Analysis System* (SAS) è un metodo strutturato ideato per valutare sistematicamente le vulnerabilità e supportare scelte progettuali che favoriscano la resilienza. Questo approccio si fonda su tre strategie difensive fondamentali:

- **Resistenza:** La capacità del sistema di respingere gli attacchi o di limitare l'estensione del danno iniziale sfruttando meccanismi di difesa perimetrale, cifratura e controllo degli accessi.
- **Identificazione:** La capacità di rilevare tempestivamente le intrusioni, i comportamenti anomali o i guasti strutturali in corso d'opera tramite sistemi di monitoraggio e audit.
- **Ripristino:** La capacità di isolare i nodi compromessi, attivare le infrastrutture ridondate e riattivare i servizi essenziali nel minor tempo possibile per garantire la continuità operativa.

L'analisi e l'applicazione di questo metodo operativo si articolano in quattro fasi sequenziali:

- **Capire il sistema:** Studio approfondito dell'architettura software, dei flussi di dati e delle dipendenze tecnologiche per definire il normale perimetro operativo.
- **Identificare i servizi critici:** Classificazione e mappatura dei servizi di core business che devono rimanere attivi a ogni costo, definendo le tolleranze di interruzione accettabili.

- **Simulare gli attacchi:** Analisi degli scenari di rischio tramite simulazioni controllate, penetration testing e modellazione delle minacce per testare la tenuta dei componenti.
- **Analizzare la sopravvivenza:** Valutazione finale della robustezza del sistema a fronte degli scenari simulati, per identificare i punti deboli e definire le necessarie contromisure ingegneristiche.

L'introduzione e l'integrazione di tecniche di sopravvivenza avanzate comportano un inevitabile onere finanziario e computazionale. Tuttavia, per ottimizzare l'allocazione del budget ed evitare costosi interventi di refactoring strutturale ex-post, è fortemente raccomandato implementare tali strategie fin dalle prime fasi della progettazione architettonica del software.